

SQLP 과목 III 튜닝 스타일 모의문제 · 10회차

SQL 자격검정 실전문제 3장 스타일 · 4지선다 · 총 20문항 · 원본 창작

1

인덱스를 경유해 테이블 행을 찾아갈 때 일어나는 Random 액세스, 그 한 건 한 건을 처리하는 Single Block I/O, 그리고 방문할 블록 수를 좌우하는 클러스터링 팩터의 관계에 대한 설명으로 가장 적절하지 않은 것은?

- ① 클러스터링 팩터가 좋을수록, 즉 인덱스 정렬 순서와 테이블 저장 순서가 잘 맞을수록, 같은 건수를 읽어도 찾아가야 할 테이블 블록 수가 늘어나 Single Block I/O 횟수가 오히려 증가한다.
- ② 인덱스 리프에서 얻은 ROWID로 테이블 블록을 한 건씩 찾아가는 것은 어디에 있을지 예측하기 어려운 블록을 임의로 방문하는 Random 액세스이며, 매 방문이 한 블록을 대상으로 하는 Single Block I/O(오라클의 db file sequential read)로 처리된다.
- ③ 원하는 행 여러 개가 같은 테이블 블록에 모여 있으면 한 번 읽어 캐시에 올린 블록에서 여러 ROWID를 연이어 처리할 수 있어, 실제 블록 방문 횟수가 처리한 ROWID 수보다 적어질 수 있다.
- ④ 클러스터링 팩터 값이 테이블의 블록 수에 가까울수록 인덱스 순서대로 읽어도 같은 블록을 다시 찾을 일이 적어 Random 액세스가 줄고, 총 로우 수에 가까울수록 행마다 다른 블록을 방문하게 되어 Random 액세스가 많아진다.

2

트랜잭션이 데이터를 변경할 때 남기는 Undo(세그먼트)가 롤백과 읽기 일관성 양쪽에서 하는 역할에 대한 설명으로 가장 적절한 것은?

- ① 읽기 일관성은 변경 전 이미지를 Redo 로그에서 되짚어 확보되므로, Undo 세그먼트가 부족하더라도 Redo 로그만 충분히 남아 있으면 다른 세션의 일관된 읽기가 보장된다.
- ② 커밋되지 않은 변경 블록은 Undo에만 머무르고 데이터파일의 블록에는 절대 반영되지 않으므로, 다른 세션이 디스크에서 커밋 전 변경 내용을 마주칠 일은 없다.
- ③ 트랜잭션이 블록을 바꾸기 전 이미지를 Undo 세그먼트에 적재하며, 이 Undo는 롤백 시 원래 값으로 되돌리는 데도 쓰이고 다른 세션이 변경 시작 전 시점의 일관된 데이터를 재구성해 읽는 읽기 일관성에도 함께 쓰인다.
- ④ Undo 세그먼트는 롤백만을 위한 영역이라 조회 전용(SELECT) 문에는 만들어지지도 사용되지도 않으며, 읽기 일관성은 버퍼 캐시에 남아 있는 블록의 여러 버전만으로 확보된다.

전자투표 개표 시스템의 집계 배치가 마감된 투표 40만 건을 순회하며 건마다 후보자별 득표 UPDATE 1건씩(총 40만 건)을 실행한다. 개발자가 EXECUTE IMMEDIATE로 SQL 문자열을 만들면서 선거구코드·후보자번호 같은 값을 바인드 변수가 아니라 문자열로 이어 붙여(리터럴) 동적 SQL을 조립했기 때문에, 실행마다 SQL 텍스트가 조금씩 달라진다. 8개의 집계 세션이 동시에 돌아 응답이 39초로 늘고 대기 이벤트 상위에 library cache: mutex X·cursor: pin S wait on X·shared pool latch가 잡혔다. 아래는 이 배치 세션의 비재귀·재귀 statement를 나눠 집계한 TKPROF이다. 지연의 원인을 직접 지목한 것으로 옳은 것은? (단, 옵티마이저 통계는 최신이고 Shared Pool은 여유가 충분해 에이징으로 인한 강제 재파싱은 없으며, 각 실행은 조건값만 다르고 UPDATE 구조는 동일하다.)

아래							
OVERALL TOTALS FOR ALL NON-RECURSIVE STATEMENTS							
Call	Count	CPU Time	Elapsed	Disk	Query	Current	Rows
Parse	400000	32.000	33.000	0	0	0	0
Execute	400000	5.000	6.000	0	800000	400000	400000
Fetch	0	0.000	0.000	0	0	0	0
Total	800000	37.000	39.000	0	800000	400000	400000
Misses in library cache during parse: 396000							
OVERALL TOTALS FOR ALL RECURSIVE STATEMENTS							
Call	Count	CPU Time	Elapsed	Disk	Query	Current	Rows
Parse	800000	6.000	6.000	0	0	0	0
Execute	800000	8.000	8.000	0	6400000	0	0
Fetch	1600000	4.000	4.000	0	9600000	0	1600000
Total	3200000	18.000	18.000	0	16000000	0	1600000
Misses in library cache during parse: 30							

- ① 재귀 statement의 Query 논리 읽기가 Execute 640만·Fetch 960만을 합쳐 1,600만에 이르고 Fetch만 160만 행을 디렉터리에서 퍼올려 재귀 Total 18.0초를 쓰는 반면 비재귀 Execute는 Query 80만뿐이므로, 이 디렉터리 논리 읽기가 병목이다. DB 버퍼 캐시를 키워 캐시 적중으로 돌리면 파싱 지연이 사라진다.
- ② 비재귀 Parse 400,000 : Execute 400,000 = 1:1이라 실행 한 번마다 파싱 콜이 한 번씩 붙고 Parse CPU 32.0초가 Execute CPU 5.0초의 6배를 넘으므로, 커서를 쥐지 못한 소프트파싱 폭주가 병목이다. session_cached_cursors와 커서 홀드로 Parse 콜을 없애면 라이브러리 캐시 mutex 대기과 39초가 함께 줄어든다.
- ③ 라이브러리 캐시 미스가 396,000건으로 비재귀 Parse 40만 콜의 거의 전부여서 매 실행이 하드파싱이고, 그 하드파싱이 디렉터리를 뒤지며 재귀 Call 320만을 유발한 것이 원인이다. 동적 SQL의 리터럴을 바인드 변수로 바꿔 정적 SQL로 커서를 공유시키면 하드파싱이 소프트파싱으로 바뀌어 39초가 해소된다.
- ④ Execute의 Query 800,000·Current 400,000이 40만 행 갱신에 행당 Query 2블록·Current 1블록을 쓰고 Execute CPU 5.0초·Elapsed 6.0초가 붙은 것이 실제 갱신 작업의 골격이며, 재귀 Total 18.0초도 이 갱신을 뒷받침하는 부수 비용이므로, UPDATE 실행계획을 다시 짜 실행 속을 최적화하면 39초가 해소된다.

4

옵티마이저가 실행 전에 추정하는 예상 실행계획과, 실제로 수행된 실행계획을 확인하는 방법, 그리고 계획을 고정해 안정성을 확보하는 수단(SQL Plan Baseline·Stored Outline)에 대한 설명으로 가장 적절한 것은?

- ① EXPLAIN PLAN이나 DBMS_XPLAN.DISPLAY로 보는 것은 옵티마이저가 실행 전에 추정한 예상 계획이라 바인드 값이나 실제 데이터 분포에 따라 실행 시점의 계획과 달라질 수 있으므로, 실제로 수행된 계획은 DBMS_XPLAN.DISPLAY_CURSOR로 커서 캐시에서 확인해야 정확하다.
- ② SQL Plan Baseline이나 Stored Outline을 적용하면 옵티마이저가 예상 계획을 산출하는 과정 자체가 생략되고 등록된 계획을 그대로 꺼내 쓰므로, 하드파싱은 물론 후보 계획을 고르는 옵티마이징 비용까지 사라지고 DISPLAY로 뽑은 예상 계획이 실행 시점 계획과 어긋날 여지도 없어진다.
- ③ EXPLAIN PLAN은 대상 SQL을 실제로 한 번 수행하면서 각 단계를 거친 실제 처리 건수와 소요 시간을 측정해 PLAN_TABLE에 적재하고 DBMS_XPLAN.DISPLAY가 그 행을 읽어 보여 주므로, 여기서 얻은 계획은 바인드 값이 달라지더라도 실행 시점의 실제 계획과 그대로 일치한다.
- ④ DBMS_XPLAN.DISPLAY_CURSOR는 커서 캐시에 올라온 계획을 보여 주는 것은 물론 아직 실행되지 않아 커서가 없는 SQL도 SQL_ID나 SQL 텍스트만 넘기면 옵티마이저를 불러 예상 계획을 산출해 주므로, EXPLAIN PLAN과 PLAN_TABLE을 거치지 않고도 실행 전 계획 확인이 가능하다.

5

해양관측 시스템에서 관측정점(약 400건)을 드라이빙해 부이관측값(약 5억 건)을 NL 조인으로 200만 행 뽑는 SQL의 TKPROF이다. 물리 읽기(Disk)가 작고 캐시 적중률이 높으니 빠를 것으로 봤는데 응답이 40초를 넘겨 느리다는 신고가 들어왔다. 아래 트레이스에 대한 해석으로 가장 적절한 것은? (단, 옵티마이저 통계는 최신이고 CPU 자원의 외부 경합은 없으며, 부이관측값의 인덱스 부이관측값_N1을 경유하는 NL 조인으로만 수행되고 조인 대상 블록은 대부분 이미 버퍼 캐시에 올라와 있어 물리 I/O가 작다.)

아 래							
Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	1	0.010	0.010	0	0	0	0
Execute	1	0.000	0.000	0	0	0	0
Fetch	2001	32.000	40.000	100000	5000000	0	2000000
Total	2003	32.010	40.010	100000	5000000	0	2000000
Rows	Row Source Operation						
2000000	NESTED LOOPS (cr=5000000 pr=100000 pw=0 time=39000000 us)						
400	TABLE ACCESS FULL 관측정점 (cr=50 pr=0 pw=0 time=1000 us)						
2000000	TABLE ACCESS BY INDEX ROWID 부이관측값 (cr=4999950 pr=100000 pw=0 time=32000000 us)						
2000000	INDEX RANGE SCAN 부이관측값_N1 (cr=3200000 pr=0 pw=0 time=14000000 us)						

- ① BCHR이 98.0%로 높아 물리 읽기(Disk 100,000)는 논리 읽기의 2%뿐이고, CPU 32초가 40초의 80%다. 병목은 디스크가 아니라 NL 조인이 부이관측값을 200만 번 방문하며 쌓은 논리 읽기 500만이 CPU를 태우는 것이므로, 조인 방식·액세스 경로를 바꿔 논리 읽기 횟수 자체를 줄여야 한다.
- ② 물리 읽기 Disk 100,000이 전부 부이관측값 노드(pr=100,000)에 몰려 있고 그 노드가 time 32,000,000us로 전체 39초의 대부분을 쓰고 있으므로, 캐시에서 못 찾아 디스크를 때린 이 단일 블록 읽기가 40초의 뿌리다. 버퍼 캐시를 키워 물리 읽기를 캐시 적중으로 돌리면 응답이 최적화된다.
- ③ Elapsed 40.000 - CPU 32.000 = 8.0초가 디스크 100,000블록을 기다린 물리 읽기 대기이고, 인덱스 스캔은 pr=0인데 부이관측값 노드만 pr=100,000을 안고 있어 대기가 그쪽에 몰렸으므로 I/O 바운드다. 병렬도(DOP)를 높여 디스크 읽기를 나눠 주면 40초가 개선된다.
- ④ 400건짜리 관측정점과 맞물려 쏟아진 결과 200만 행이 해시·정렬 영역을 넘겨 temp로 흘린 spill이 원인이고, NESTED LOOPS의 time 39,000,000us와 Fetch 2001회는 그 temp를 되읽으며 왕복한 흔적이므로, PGA를 키워 이 spill을 없애면 40초가 사라진다.

전기시설 안전점검 관리 시스템의 안전점검 테이블(약 940만 건)에는 결합 인덱스 점검_IX = (관할지사코드, 점검일자)가 있다. 점검을 수행한 관할지사명을 함께 얻기 위해 관할지사 테이블(기본키 관할지사_PK = 관할지사코드)과 조인하는 다음 SQL의 실행계획이다. 이 실행계획에 대한 설명으로 가장 적절한 것은?

아 래

```
SELECT p.점검번호, p.점검일자, p.시설명, g.관할지사명
FROM 안전점검 p, 관할지사 g
WHERE p.관할지사코드 = 'D07'
      AND p.점검일자 >= DATE '2026-06-01'
      AND p.점검일자 < DATE '2026-07-01'
      AND p.판정결과 = '부적합'
      AND g.관할지사코드 = p.관할지사코드;
```

Id	Pid	Operation	(Cost	Card	Bytes)
0	-	SELECT STATEMENT	(412	1900	121600)
1	0	NESTED LOOPS	(412	1900	121600)
2	1	TABLE ACCESS BY INDEX ROWID 안전점검	(405	1900	79800)
3	2	INDEX RANGE SCAN 점검_IX	(22	7200	)
4	1	TABLE ACCESS BY INDEX ROWID 관할지사	(1	1	22)
5	4	INDEX UNIQUE SCAN 관할지사_PK	(0	1	)

- ① WHERE의 세 조건(관할지사코드 등치·점검일자 범위·판정결과 등치)이 모두 점검_IX의 인덱스 액세스 조건으로 INDEX RANGE SCAN(Cost 22) 단계에서 함께 처리되어 스캔 범위를 결정하며, 그렇게 좁혀 남은 건수가 Card 7,200으로 찍힌 것이다.
- ② INDEX RANGE SCAN이 반환한 Card 7,200이 곧 이 SQL의 최종 결과 건수이며, 상위 TABLE ACCESS BY INDEX ROWID의 Card 1,900은 940만 건 테이블에서 실제로 읽어 온 블록 수를 표시한 값이므로, 테이블 액세스 단계에서는 건수 감소가 일어나지 않는다.
- ③ 점검일자가 범위(>=, <) 조건이므로, 선두 칼럼 관할지사코드의 등치도 인덱스 액세스 조건이 되지 못하고 스캔 후 필터로만 처리되며, 그 탓에 INDEX RANGE SCAN이 Cost 22·Card 7,200으로 넓게 훑은 뒤 걸러내는 모습이 된 것이다.
- ④ 관할지사코드(등치)와 점검일자(범위)까지가 점검_IX의 인덱스 액세스 조건으로 INDEX RANGE SCAN에서 처리(Card 7,200)되고, 인덱스에 없는 판정결과는 TABLE ACCESS BY INDEX ROWID 단계 필터로 걸려져 Card가 1,900으로 줄어든다.

현책방 체인의 중고도서 거래 시스템에서 중고도서판매 테이블(약 4,000만 건, 블록당 약 100행 → 약 40만 블록)을 특정 장르코드 조건으로 조회한다. 장르 컬럼의 인덱스 중고도서판매_N1로 25만 건을 뽑는데, 인덱스를 타는데도 30초가 걸린다. 아래 TKPROF에서 지연의 원인을 직접 지목한 것으로 옳은 것은?

아 래							
Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	1	0.004	0.004	0	0	0	0
Execute	1	0.000	0.000	0	0	0	0
Fetch	501	3.500	29.996	50000	246250	0	250000
Total	503	3.504	30.000	50000	246250	0	250000
Rows	Row Source Operation						
250000	TABLE ACCESS BY INDEX ROWID 중고도서판매 (cr=246250 pr=50000 pw=0 time=29600000 us)						
250000	INDEX RANGE SCAN 중고도서판매_N1 (cr=1250 pr=0 pw=0 time=350000 us)						

- ① 병목은 INDEX RANGE SCAN이다. 4,000만 건에서 25만 건의 ROWID를 순서대로 내주는 이 스캔이 cr 1,250·pr 0·time 350,000us를 쓰며 이 SQL에서 리프를 통째로 훑는 유일한 탐색 작업인데, 리프 블록이 조각나 재탐색이 늘어진 탓에 Fetch 29.996초의 대부분이 여기서 났으므로, 인덱스를 리빌드해 리프를 다지면 30초가 준다.
- ② Parse 0.004초·Execute 0.000초로 파싱과 실행은 사실상 공짜이고 시간이 붙은 곳은 Fetch 콜뿐인데, 25만 행을 500건씩 나눠 담느라 서버 왕복이 501번 일어났고 CPU 3.5초에 비해 Elapsed 29.996초가 압도적으로 큰 것이 곧 그 왕복 대기이므로, 배열 인출 크기(현재 500)를 키우면 지연이 사라진다.
- ③ 물리 읽기 Disk 50,000이 논리 읽기 246,250의 5분의 1이고 인덱스 노드는 pr=0이라 그 전량이 테이블 노드(pr=50,000)에 몰려 있어, 29.996초에서 CPU 3.5초를 뺀 26.5초 대기의 원인이 이 물리 읽기다. 버퍼 캐시를 키워 50,000을 캐시 적중으로 돌리면 245,000블록의 논리 랜덤 액세스 비용까지 사라져 손익분기점 문제가 해소된다.
- ④ 테이블 액세스 자체 블록 I/O가 245,000으로 ROWID 수 250,000과 거의 같아 클러스터링 팩터가 최악이다. 이 245,000이 cr의 대부분과 time 29.6초를 차지하고 40만 블록의 61.25%를 단일 블록 랜덤 읽기로 훑어 손익분기점을 넘었으므로, 커버링 인덱스로 테이블 액세스 자체를 없애는 것이 답이다.

Index Unique Scan과 Index Range Scan의 성립 조건, 그리고 Range Scan에서 실제로 훑는 리프 구간(스캔 범위)이 무엇으로 정해지는지에 대한 설명으로 가장 적절하지 않은 것은?

- ① Index Unique Scan은 유일 인덱스에 등치 조건이 걸려 조건을 만족하는 행이 최대 한 건일 때 성립하며, 하나의 리프 엔트리를 찾으면 더 읽을 것이 없어 곧바로 스캔을 끝낸다.
- ② Index Range Scan이 실제로 훑는 리프 구간의 폭은 인덱스에 존재하는 선두 컬럼 조건과는 무관하게, 인덱스에 없는 일반 필터 조건이 많이 걸릴수록 좁아지며 그 필터들이 스캔의 시작점과 종료점을 결정한다.
- ③ Index Range Scan의 스캔 시작점과 종료점은 인덱스 선두 컬럼에 걸린 액세스 조건이 정하며, 그 두 지점 사이의 리프를 수평으로 훑는다. 액세스 조건에 걸리지 못한 조건은 스캔 범위를 줄이지 못하고 훑는 도중 필터로만 걸러진다.
- ④ 같은 인덱스라도 선두 컬럼에 등치가 아닌 범위 조건이 걸리면 시작·종료 폭이 넓어져 훑는 리프가 늘고, 뒤쪽 컬럼의 조건은 시작점을 더 좁히지 못해 스캔 범위가 커질 수 있다.

인덱스 설계에서 비트맵 인덱스를 언제 쓰고 무엇을 조심해야 하는지 — 낮은 카디널리티·집계 위주 환경에서의 장점과 DML 부하 측면의 약점 — 에 대한 설명으로 가장 적절하지 않은 것은?

- ① 비트맵 인덱스는 값의 종류가 적은(낮은 카디널리티) 컬럼에서 각 값마다 비트맵을 만들어, 여러 조건을 AND·OR 비트 연산으로 결합해 집계하기 좋으므로 갱신이 드문 OLAP·DW 환경에 잘 맞는다.
- ② 비트맵 인덱스가 DML에 불리한 까닭은 오로지 인덱스 세그먼트가 커져 재구성 비용이 크기 때문이므로, 낮은 카디널리티 컬럼에만 만들어 세그먼트를 작게 유지하면 DML 부하 문제는 해소되어 OLTP에서도 무리 없이 쓸 수 있다.
- ③ 하나의 비트맵 조각(키-값)이 다수의 행에 대응하므로, 한 행만 바꾸는 DML이라도 그 값에 묶인 비트맵에 락이 걸려 같은 비트맵을 갱신하려는 다른 트랜잭션이 대기할 수 있어, 동시 DML이 잦은 OLTP에는 부담이 크다.
- ④ 인덱스 설계는 조회 패턴만이 아니라 대상 컬럼의 카디널리티와 DML 빈도를 함께 따져야 하며, 같은 컬럼이라도 집계 위주 DW에서는 비트맵이, 갱신 잦은 OLTP에서는 B*Tree가 더 유리할 수 있다.

상표출원 관리 시스템에서, 심사관 대시보드에 가장 최근에 접수된 출원 20건만 최신순으로 띄운다. 상표출원 테이블(약 6,400만 건)에는 출원일시 기준 인덱스 출원일시_IX = (출원일시)가 있고, 아래는 그 화면 쿼리의 실행계획이다. 이 플랜의 Top-N 부분범위처리에 대한 설명으로 가장 적절하지 않은 것은?

아래

```
SELECT *
FROM ( SELECT 출원번호, 출원일시, 상표명, 출원인명
      FROM 상표출원
      ORDER BY 출원일시 DESC )
WHERE ROWNUM <= 20;
```

Id	Operation	Name
0	SELECT STATEMENT	
* 1	COUNT STOPKEY	
2	VIEW	
3	TABLE ACCESS BY INDEX ROWID	상표출원
4	INDEX FULL SCAN DESCENDING	출원일시_IX

- ① 이 플랜은 상표출원 6,400만 건의 출원일시를 전부 정렬한 뒤 그중 앞 20건을 잘라내는 방식이며, 정렬을 위해 테이블 전체를 끝까지 읽는다.
- ② Id 1 COUNT STOPKEY가 ROWNUM 한도(20)에 도달하는 즉시 하위 노드에 정지 신호를 보내므로, 20건을 채우면 그 아래 스캔이 더 진행되지 않고 멈춘다.
- ③ 출원일시_IX를 역방향(INDEX FULL SCAN DESCENDING)으로 스캔하면 최신순 순서가 그대로 나오므로, 플랜에 별도의 SORT ORDER BY 노드 없이 인덱스 순서를 최신순으로 사용한다.
- ④ 최신 20건만 필요한 부분범위처리이므로, 인덱스를 내림차순으로 앞에서부터 훑어 20건이 차는 지점까지만 접근하고 나머지 방대한 구간은 건드리지 않는다.

11

해시 조인(Hash Join)에서 두 입력 중 어느 쪽이 Build Input이 되는지, 그 선택이 조인 순서·해시 영역(Hash Area)·비용과 어떻게 맞물리는지에 대한 설명으로 가장 적절한 것은?

- ① FROM 절에 먼저 적은 테이블이 Build Input이 되어 해시 테이블로 만들어지고 뒤에 적은 쪽이 Probe로 흐르므로, 큰 테이블을 FROM 절 뒤로 미뤄 적기만 하면 Build Input이 작아져 해시 영역에 담길 확률이 올라가고 조인 순서도 적어 준 표기 순서대로 고정된다.
- ② Build Input이 해시 영역에 다 담기지 못할 만큼 크면 옵티마이저는 실행 도중 해시 조인을 접고 소트 머지 조인으로 자동 전환해, 두 집합을 조인 키로 정렬한 뒤 순서대로 맞대어 처리하므로 두 입력을 분할해 Temp에 내려쓰는 작업 없이 조인이 끝난다.
- ③ 옵티마이저는 두 입력 중 더 작다고 추정한 집합을 Build Input으로 골라 해시 테이블을 만들며, 이 선택은 어느 쪽을 먼저 처리할지의 조인 순서와 맞물려 결정된다. Build Input이 해시 영역에 다 담기면 Temp 경유 없이 조인이 끝나므로 작은 집합을 Build로 잡는 편이 비용에 유리하다.
- ④ 두 입력이 모두 대량이면 각 행마다 인덱스를 타고 상대 테이블을 찾아가는 NL 조인보다 한 번의 해시 테이블 구축으로 끝내는 해시 조인의 비용이 낮으므로, 해시 영역 크기나 Temp 경유 여부와 무관하게 조인 대상이 크기만 하면 어느 쪽을 Build로 잡든 해시 조인이 비용상 우위로 선택된다.

12

SELECT 절의 스칼라 서브쿼리(Scalar Subquery)를 옵티마이저가 조인(주로 아우터 조인)으로 바꾸는 쿼리 변환과, 그 변환이 스칼라 서브쿼리 캐싱과 맺는 관계에 대한 설명으로 가장 적절하지 않은 것은?

- ① SELECT 절 스칼라 서브쿼리는 옵티마이저가 조인으로 풀어낼 수 있으며(스칼라 서브쿼리 언네스팅), 변환되면 메인 행마다 서브쿼리를 반복 수행하던 것을 집합 기반 조인으로 처리하게 된다.
- ② 스칼라 서브쿼리가 조인으로 변환되면 스칼라 서브쿼리 캐싱도 함께 걸려, 조인으로 처리하면서 동시에 입력값 캐싱까지 작동해 중복 값에 대한 재수행이 이중으로 줄어든다.
- ③ 스칼라 서브쿼리는 메인 행마다 결과가 1건이어야 하므로 조인으로 변환할 때 아우터 조인 형태가 되며, 서브쿼리가 한 행에 대해 2건 이상을 반환하면 오류가 나는 '1건 보장' 의미는 변환 뒤에도 유지된다.
- ④ 변환 여부는 비용 기반으로 판단되므로, 서브쿼리 입력값의 구별 값이 적고 반복이 많아 캐싱이 잘 듣는 상황에서는 조인으로 변환하지 않고 스칼라 서브쿼리 캐싱으로 두는 편이 유리할 수 있다.

13

문화재 보수 관리 시스템의 관리자 대시보드가 보수공사내역 테이블(약 5,300만 건, 문화재구분 6종)에서 문화재구분별 보수비용 합계를 조회한다. 같은 화면이 초 단위로 반복 호출돼 집계를 매번 다시 수행하는 부하가 커서, 아래처럼 /*+ RESULT_CACHE */ 힌트를 넣고 문화재구분을 바인드 변수로 발행하도록 고쳤다. 결과 캐시(Result Cache)의 동작과 커서 공유·무효화에 대한 설명으로 가장 적절하지 않은 것은?

아래

```
-- 문화재구분별 보수비용 합계를 결과 캐시로 조회
SELECT /*+ RESULT_CACHE */ SUM(보수비용)
FROM   보수공사내역
WHERE  문화재구분 = :b1;
-- :b1 에 '국보', '보물', '사적' 값을 바꿔 반복 실행

-- 야간 배치가 보수공사내역에 신규 공사분을 INSERT/UPDATE 하고 커밋한다
```

- ① 결과 캐시는 질의 결과 자체를 SGA의 Result Cache 영역에 저장해, 동일 질의의 다음 실행에서 블록을 다시 읽거나 집계를 재수행하지 않고 저장된 결과를 곧바로 돌려준다.
- ② 캐시가 참조하는 보수공사내역에 DML이 커밋되면 그 테이블에 의존하는 결과 캐시 항목이 무효화되어, 다음 실행 때 결과를 다시 계산해 새로 캐싱한다.
- ③ 결과 캐시는 커서 공유와 다른 계층이다 — 커서 공유는 라이브러리 캐시의 파싱된 커서(실행계획)를 재사용하는 것이고 결과 캐시는 계산된 결과값을 재사용하는 것이라, 결과 캐시가 적중해도 커서를 찾는 소프트파싱 자체는 일어난다.
- ④ 결과 캐시는 SQL 텍스트가 같으면 캐시를 공유하므로, :b1에 '국보'로 채운 실행과 '보물'로 채운 실행은 텍스트가 동일한 이상 같은 캐시 결과를 재사용해 한 번만 계산된다.

비용 기반 옵티마이저(CBO)가 통계를 바탕으로 카디널리티·선택도·조인 카디널리티를 추정하는 원리에 대한 설명으로 가장 적절한 것은?

- ① 선택도(selectivity)는 조건식에서, 카디널리티(추정 행 수)는 테이블 통계에서 각각 따로 산출되는 독립 지표라, 선택도가 낮게 나와도 카디널리티는 높게 추정될 수 있고 그 반대도 가능해 두 값 사이에 정해진 산술 관계가 없으며 통계를 새로 수집해도 마찬가지다.
- ② 히스토그램은 컬럼 값의 분포를 구간별 빈도로 담으므로, 히스토그램이 있는 컬럼을 조인 키로 쓰면 단일 조건 선택도는 물론 두 집합을 맞물릴 때의 조인 카디널리티 추정까지 정확해지고, 그 추정에 따라 NL·해시·소트 머지 같은 조인 방식과 조인 순서도 최적으로 결정된다.
- ③ 조인 카디널리티는 두 집합의 카디널리티와 조인 컬럼의 값 분포로 추정되며, 조인 컬럼의 구별 값 수(NDV)가 그 추정 정확도를 크게 좌우한다. 그래서 조인 컬럼 통계가 실제와 어긋나면 각 테이블의 단일 조건 선택도가 정확해도 조인 결과 건수 추정이 빗나갈 수 있다.
- ④ 통계가 최신일수록 카디널리티 추정 오차가 줄어들 뿐 아니라 옵티마이저가 비교해 볼 후보 계획이 줄어 하드파싱에 드는 SQL 파싱 시간까지 함께 단축되므로, 통계 갱신은 실행계획 품질과 파싱 성능이라는 두 축을 한 번에 끌어올리는 가장 값싼 조치가 된다.

전략 석유비축 관리의 야간 배치 하루치 유류 입출고(약 8,000만 행)를 수집 테이블에서 비축유입출고(약 12억 행)로 대량 적재한다. 적재 시간을 줄이려고 아래처럼 APPEND 힌트와 병렬 DML을 적용했다. 비축유입출고 세그먼트는 NOLOGGING이고 DB는 force logging이 아니며, 이 테이블에는 비유니크 인덱스 3개가 걸려 있다. 이 대량 DML 튜닝에 대한 설명으로 가장 적절하지 않은 것은?

아래

— 일일 유류 입출고를 비축유입출고에 대량 적재하는 야간 배치
ALTER SESSION ENABLE PARALLEL DML;

INSERT /*+ APPEND PARALLEL(t, 4) */ INTO 비축유입출고 t
SELECT * FROM 입출고_수집;

COMMIT;

- ① INSERT /*+ APPEND */ 는 Direct Path Insert로 동작해, HWM 위쪽 새 블록에 데이터를 기록하고 버퍼 캐시와 freelist 탐색을 우회하므로 대량 적재 속도가 빨라진다.
- ② 세그먼트가 NOLOGGING이고 DB가 force logging이 아니어야 Direct Path Insert가 데이터에 대한 redo를 최소화한다 — 이 조건이 아니면 direct path여도 redo가 정상적으로 쌓이고, 일반(conventional) 인서트에는 nologging이 redo 절감 효과를 주지 못한다.
- ③ ALTER SESSION ENABLE PARALLEL DML 후 병렬 INSERT를 수행하면 여러 PX 서버가 각자 자기 익스텐트에 direct path로 적재하며, 이때 대상 테이블은 배타 모드로 잠겨 커밋 전까지 같은 테이블에 대한 다른 DML은 대기한다.
- ④ Direct Path Insert도 인덱스는 일반 인서트처럼 행을 넣을 때마다 즉시 리프에 반영하므로, 적재 전에 인덱스를 unusable로 돌렸다가 적재 후 재생성하는 방식은 대량 적재에서 이득이 없다.

전자수입인지 발행 관리의 수입인지발행(약 2억 6천만 건)은 발행일자(DATE)로 월 단위 INTERVAL RANGE 파티셔닝한 테이블이다(아래 DDL). 각 선택지는 SELECT SUM(발행금액) FROM 수입인지발행 WHERE ... 형태의 조회가 어떤 파티션을 접근하는지, 또는 인터벌 파티션이 어떻게 생성되는지를 설명한다. Partition Pruning과 인터벌 자동 생성이 설명대로 작동한다고 볼 때 가장 적절한 것은?

아 래

```
CREATE TABLE 수입인지발행 (
  발행번호 NUMBER,
  발행일자 DATE,
  청사코드 NUMBER,
  인지종류 VARCHAR2(20),
  발행금액 NUMBER
)
PARTITION BY RANGE (발행일자)
INTERVAL (NUMTOYMINTERVAL(1, 'MONTH'))
(
  PARTITION p_init VALUES LESS THAN (DATE '2026-01-01')
);
```

- ① INTERVAL 파티셔닝은 NUMTOYMINTERVAL(1, 'MONTH') 규칙을 읽어 테이블 생성 시점에 p_init 다음 12개월치 월별 파티션을 미리 만들어 두므로, 2026년 어느 달의 발행일자가 들어와도 이미 준비된 파티션에 바로 저장되고 운영 중에 DDL을 다시 손댈 필요가 없다.
- ② WHERE TO_CHAR(발행일자, 'YYYYMM') = '202603' 은 파티션 키인 발행일자를 월 문자열로 바꿔 3월을 겨냥한 조건이므로, p_init 뒤로 인터벌이 만들어 낸 2026년 3월 파티션 하나만 프루닝되어 2억 6천만 건 중 그 파티션만 읽고 나머지 달은 건드리지 않는다.
- ③ WHERE 발행금액 >= 100000 은 NUMBER 컬럼에 값의 범위를 지정한 조건이고 이 테이블이 PARTITION BY RANGE로 만들어졌으므로 RANGE 프루닝이 그대로 작동해, 발행금액이 큰 행이 몰린 상위 파티션들만 골라 읽고 2억 6천만 건의 나머지는 건너뛰다.
- ④ WHERE 발행일자 >= DATE '2026-03-01' AND 발행일자 < DATE '2026-04-01' 은 파티션 키에 가공이 없는 범위 조건이라 2026년 3월 구간 파티션만 접근하며, 그 파티션이 없던 상태에서 해당 월 데이터가 처음 insert되면 인터벌 규칙으로 자동 생성된다.

17

송전수급 정산 야간 배치가 급전지시(약 4억 1천만 건)와 발전기실적(약 2억 8천만 건)을 발전기ID로 조인해 발전기별 정산금액을 집계한다. 두 테이블 모두 대형이며, 어느 쪽도 발전기ID 기준 파티션이 아니다. 아래 병렬 실행계획에서 두 테이블이 병렬 서버(Slave) 사이에 어떤 분배 방식으로 조인되는지 직접 지목한 설명으로 가장 적절한 것은?

아래

Id	Operation	Name	Pstart	Pstop	TQ	IN-OUT
0	SELECT STATEMENT					
1	PX COORDINATOR					
2	PX SEND QC (RANDOM)	:TQ10002			Q1,02	P->S
3	HASH GROUP BY				Q1,02	PCWP
* 4	HASH JOIN BUFFERED				Q1,02	PCWP
5	PX RECEIVE				Q1,02	PCWP
6	PX SEND HASH	:TQ10000			Q1,00	P->P
7	PX BLOCK ITERATOR				Q1,00	PCWC
8	TABLE ACCESS FULL	급전지시			Q1,00	PCWP
9	PX RECEIVE				Q1,02	PCWP
10	PX SEND HASH	:TQ10001			Q1,01	P->P
11	PX BLOCK ITERATOR				Q1,01	PCWC
12	TABLE ACCESS FULL	발전기실적			Q1,01	PCWP

- ① 소형 쪽을 PX SEND BROADCAST로 모든 병렬 서버에 복제하고 대형 쪽은 재분배 없이 각 서버가 자기 블록 범위만 읽어 조인하는 broadcast 분배이며, 그 복제가 Id 6에서 일어난다.
- ② 조인 키 발전기ID의 해시로 급전지시(Id 8)와 발전기실적(Id 12)을 양쪽 모두 재분배(hash-hash) 한 뒤 각 서버가 자기 버킷 범위의 행만 조인하며, 그 재분배가 Id 6·Id 10의 PX SEND HASH에서 각각 일어난다.
- ③ 두 테이블이 발전기ID로 동일하게 파티셔닝돼 있어 재분배 없이 대응 파티션 집합만 로컬로 조인하는 (full) 파티션 와이즈 조인이며, Id 8·Id 12가 같은 PX PARTITION 아래에서 읽힌다.
- ④ 대형 급전지시만 파티션 단위로 각 서버가 읽고, 발전기실적을 급전지시의 파티션 경계에 맞춰 PX SEND PARTITION (KEY)로 재분배하는 부분(partial) 파티션 와이즈 조인이다.

18

분석함수(윈도우 함수)의 프레임·순위 함수와 집합 연산자(UNION/UNION

ALL/INTERSECT/MINUS)의 정렬·중복 처리에 대한 설명으로 가장 적절하지 않은 것은?

- ① UNION은 두 집합을 합친 뒤 중복을 없애기 위해 SORT UNIQUE(또는 HASH UNIQUE) 작업이 따르지만, UNION ALL은 중복 제거를 하지 않아 그 정렬·해시 없이 두 집합을 그대로 이어 붙인다.
- ② 분석함수의 윈도우 프레임에서 ROWS와 RANGE는 둘 다 물리적 행 개수로 범위를 세는 방식이라, ROWS BETWEEN을 RANGE BETWEEN으로 바꿔도 같은 결과가 나온다.
- ③ RANK와 DENSE_RANK는 동순위(tie)를 허용하되 그다음 순위 부여가 달라, RANK는 동순위 뒤 순위를 건너뛰고(1,1,3) DENSE_RANK는 건너뛰지 않으며(1,1,2), ROW_NUMBER는 동순위여도 유일한 일련번호를 매긴다.
- ④ INTERSECT와 MINUS도 UNION처럼 중복 제거를 수반해 정렬(또는 해시) 작업이 따르며, 결과로 중복이 제거된 유일 집합을 돌려준다.

19

후원금영수증 발급 시스템의 후원금영수증(약 3,100만 건)에 단체 가·나·다의 누적 후원금액이 각각 120,000·240,000·360,000원(합 720,000)으로 들어 있다. 집계 세션 TX2가 하나의 트랜잭션 안에서 SELECT SUM(후원금액) FROM 후원금영수증을 두 번 발행한다(t1, t3). 그 사이 t2에 다른 세션 TX1이 나·다의 후원금액을 갱신하고 커밋한다. TX2의 두 번째 SELECT(t3)가 반환하는 SUM(후원금액)으로 적절한 것은? (단, 오라클이며 격리수준은 기본값 READ COMMITTED이고 TX2는 SERIALIZABLE로 올리지 않았으며, 관련 Undo는 정상적으로 남아 있다.)

아래

시각	TX1 (갱신 · 커밋)	TX2 (집계 SELECT × 2, 한 트랜잭션)
t1		SELECT SUM(후원금액) FROM 후원금영수증; → 첫 번째 집계
t2	UPDATE 후원금영수증 SET 후원금액=후원금액+60000 WHERE 단체='나'; (나 240000→300000) UPDATE 후원금영수증 SET 후원금액=후원금액+40000 WHERE 단체='다'; (다 360000→400000) COMMIT;	
t3		SELECT SUM(후원금액) FROM 후원금영수증; → 두 번째 집계 (이 값을 구함)

초기: 가=120000, 나=240000, 다=360000 (합 720000)

- ① 820,000
- ② 780,000
- ③ 760,000
- ④ 720,000

20

산불감시 관제 시스템(SQL Server)의 산불감지이벤트(약 4,700만 건)는 READ_COMMITTED_SNAPSHOT(RCSI)을 ON으로 설정해 운영한다. 세션 52가 특정 감지구역 행을 갱신하는 트랜잭션을 열어(미커밋) X 잠금을 쥔 사이, 조회 세션 67이 같은 행을 SELECT하고, 갱신 세션 58이 같은 행을 UPDATE하려 한다. 이때 sys.dm_tran_locks 등을 조회한 결과가 아래와 같다. 이 상황에 대한 해석으로 가장 적절한 것은?

아래

[sys.dm_tran_locks / 요청 상태 - 산불감지이벤트 (RCSI = ON)]				
session_id	request_mode	resource_type	request_status	작업
52	X	KEY	GRANT	UPDATE (미커밋, X 보유)
67	(없음)	-	-	SELECT (버전 읽기, 잠금 요청 없음)
58	X	KEY	WAIT	같은 행 UPDATE 시도 → 52 대기

- * RCSI = ON : READ COMMITTED 하에서 읽기가 행 버전을 사용
- * KEY = 행(키) 잠금, X = 배타 잠금

- ① RCSI를 켜면 SELECT도 갱신 세션과 마찬가지로 행에 배타 잠금을 걸어 읽는 버전을 고정하므로, 67이 읽는 동안에는 52의 UPDATE가 오히려 차단된다.
- ② RCSI 하에서는 UPDATE끼리도 행 버전을 읽어 처리하므로 58은 52와 잠금 경합 없이 같은 행을 동시에 갱신할 수 있고, 나중 것이 앞의 것을 덮어쓴다.
- ③ RCSI 하의 SELECT(67)는 X 잠금을 쥔 52를 기다리지 않고 버전 저장소(tempdb)의 마지막 커밋된 행 버전을 읽으므로 블로킹이 없다. 다만 갱신 세션끼리(58↔52)는 여전히 X 잠금으로 경합해 58이 대기한다.
- ④ 조회 세션과 갱신 세션이 함께 tempdb를 사용하므로, 두 세션이 tempdb를 같이 쓴다는 사실 자체가 낙관적 동시성 제어가 작동 중이라는 증거이며, 따라서 잠금 대기(LCK_M_*)가 나타나지 않아야 정상이다.

정답 및 해설

■ 정답 모아보기

1. ①	2. ③	3. ③	4. ①	5. ①
6. ④	7. ④	8. ②	9. ②	10. ①
11. ③	12. ②	13. ④	14. ③	15. ④
16. ④	17. ②	18. ②	19. ①	20. ③

문제 1. 정답 ①

클러스터링 팩터가 '좋다/나쁘다'가 블록 방문 횟수를 어느 방향으로 움직이는지를 갈라 봐야 합니다.

클러스터링 팩터(CF)의 범위

좋은(작음) : CF ≈ 테이블 블록 수 → 인덱스 순서와 테이블 저장 순서 일치
→ 인접 ROWID가 같은 블록에 몰림 → 블록 방문 횟수 적음 → Random 액세스 적음
나쁜(큼) : CF ≈ 총 로우 수 → 행마다 다른 블록에 흩어짐
→ ROWID마다 새 블록 방문 → 블록 방문 횟수 많음 → Random 액세스 많음

클러스터링 팩터는 인덱스 정렬 순서를 따라 테이블을 읽을 때 방문하게 되는 블록 수의 척도입니다. 인덱스 순서와 테이블 저장 순서가 잘 맞으면(CF가 좋으면) 인접한 ROWID가 같은 블록에 모여 있어 한 블록을 읽는 동안 여러 행이 처리되고, 그만큼 찾아가야 할 블록 수와 Single Block I/O 횟수가 줄어듭니다.

①은 이 방향을 정반대로 뒤집었습니다. "CF가 좋을수록 방문 블록 수가 늘고 Single Block I/O가 증가한다"고 했지만, CF가 좋을수록 방문 블록 수는 오히려 감소합니다. 개선을 악화로 서술한 것입니다.

②·③·④는 옳습니다. ROWID를 통한 임의의 블록 방문이 Single Block I/O로 처리된다는 점(②), 같은 블록에 모인 행은 한 번 읽어 여러 ROWID를 처리한다는 점(③), 그리고 CF가 블록 수에 가까울수록 좋고 로우 수에 가까울수록 나쁘다는 관계(④)가 모두 랜덤 액세스의 실제 동작에 부합합니다.

선택지	판정	이유
①	×	CF가 좋을수록 인접 ROWID가 같은 블록에 몰려 방문 블록 수와 Single Block I/O가 줄어듭니다. "좋은수록 블록 방문·I/O가 는다"는 것은 개선 방향을 정반대로 뒤집은 서술입니다
②	○	리프의 ROWID로 예측 어려운 테이블 블록을 임의의 방문하는 것이 Random 액세스이며, 매 방문이 한 블록 단위 Single Block I/O(db file sequential read)로 처리됩니다
③	○	원하는 행이 같은 블록에 모여 있으면 한 번 읽은 블록에서 여러 ROWID를 처리해, 실제 블록 방문 횟수가 ROWID 수보다 적어질 수 있습니다
④	○	CF가 블록 수에 가까우면 순서가 맞아 Random 액세스가 적고, 총 로우 수에 가까우면 행마다 다른 블록을 방문해 Random 액세스가 많습니다

■ 이 문제의 핵심

1. 인덱스 경우 테이블 액세스는 ROWID로 임의의 블록을 찾아가는 Random 액세스이고, 한 건이 Single Block I/O로 처리됩니다.
2. 클러스터링 팩터는 인덱스 순서로 테이블을 읽을 때의 블록 방문 횟수 척도입니다.
3. CF가 블록 수에 가까울수록 좋고(방문 적음), 로우 수에 가까울수록 나쁩니다(방문 많음).
4. CF가 좋으면 인접 ROWID가 같은 블록에 몰려 Single Block I/O 횟수가 줄어듭니다.

■ 한 줄 정리 클러스터링 팩터가 좋으면 인접 ROWID가 같은 블록에 몰려 방문 블록 수와 Single Block I/O가 줄어드는데, "좋은수록 I/O가 늘어난다"는 ①은 그 방향을 정반대로 뒤집은 서술입니다.

문제 2. 정답 ③

Undo가 롤백 하나만 위한 것인지, 읽기 일관성까지 떠받치는지를 갈라 봐야 합니다.

트랜잭션이 블록 변경

→ 변경 전 이미지(before image)를 Undo 세그먼트에 기록

└─ 용도 1: 롤백 → 변경 전 값으로 원복

└─ 용도 2: 읽기 일관성 → 다른 세션이 "변경 시작 전 시점" 데이터를 Undo로 재구성해 읽음

(Redo 로그: 변경 후 이미지를 재적용(복구)하기 위한 것 - 읽기 일관성과 역할이 다름)

Undo 세그먼트에는 블록을 변경하기 전의 이미지가 쌓입니다. 이 정보는 트랜잭션을 되돌릴 때(롤백) 원래 값으로 복구하는데 쓰이는 동시에, 다른 세션이 아직 커밋되지 않은 변경을 건너뛰고 변경 시작 전 시점의 일관된 데이터를 재구성해 읽도록 하는 읽기 일관성의 재료로도 쓰입니다. ③은 이 두 용도를 정확히 짚었습니다.

①은 '로그'라는 단어에서 Redo를 반사적으로 떠올리게 한 함정입니다. 변경 전 이미지를 보관해 읽기 일관성을 떠받치는 것은 Undo이지 Redo가 아닙니다. Redo는 변경 후 이미지를 재적용하는 복구용입니다. ②는 더티 블록이 커밋 전에도 DBWR에 의해 데이터파일로 내려갈 수 있다는 사실을 무시했고, 그렇게 내려간 미커밋 블록도 읽는 쪽은 Undo로 되돌려 일관되게 읽습니다. ④는 SELECT도 변경 중인 블록을 만나면 Undo로 이전 시점을 재구성해 읽으므로, Undo가 조회와 무관하다는 서술이 틀렸습니다.

선택지	판정	이유
①	×	변경 전 이미지를 보관해 읽기 일관성을 떠받치는 것은 Undo이지 Redo가 아닙니다. '로그→Redo'라는 반사적 연상에 기댄 서술입니다
②	×	커밋 전 더티 블록도 DBWR에 의해 데이터파일로 내려갈 수 있으며, 읽는 쪽은 그 블록을 Undo로 되돌려 일관되게 읽습니다
③	○	변경 전 이미지를 Undo 세그먼트에 남겨 롤백 원복과, 변경 시작 전 시점을 재구성하는 읽기 일관성 양쪽에 함께 사용합니다
④	×	SELECT도 변경 중인 블록을 만나면 Undo로 이전 시점을 재구성해 읽으므로, Undo가 조회와 무관하다는 서술은 틀렸습니다

▪ 이 문제의 핵심

1. Undo 세그먼트에는 블록의 변경 전 이미지가 쌓입니다.
2. Undo는 롤백 원복과 읽기 일관성(변경 시작 전 시점 재구성) 양쪽에 쓰입니다.
3. Redo는 변경 후 이미지를 재적용하는 복구용으로, 읽기 일관성과 역할이 다릅니다.
4. 미커밋 더티 블록도 디스크로 내려갈 수 있어, 읽는 쪽이 Undo로 되돌려 일관성을 확보합니다.

▪ 한 줄 정리 Undo 세그먼트의 변경 전 이미지는 롤백과 읽기 일관성 양쪽에 쓰이며(③), 읽기 일관성을 Redo로 확보한다거나 Undo가 조회와 무관하다는 서술은 역할을 반사적으로 잘못 연결한 것입니다.

문제 3. 정답 ③

동적 SQL이 실행마다 리터럴 값을 이어 붙이므로 SQL 텍스트가 매번 달라지고, 커서는 라이브러리 캐시에서 공유되지 못합니다. 그러면 파싱은 대부분 하드파싱이 됩니다. 먼저 하드파싱율을 쟁니다.

하드파싱율 = 라이브러리 캐시 미스(비재귀) ÷ 비재귀 Parse 콜
= 396,000 ÷ 400,000 = 99.0%

→ 거의 매 실행이 계획을 새로 생성하는 하드파싱

하드파싱은 계획을 새로 만들 때마다 데이터 딕셔너리(권한·통계·객체 정보)를 조회하는 재귀 statement를 대량으로 유발합니다. 재귀 축을 봅시다.

재귀 Call 비율 = 재귀 Total 콜 ÷ 비재귀 Total 콜 = 3,200,000 ÷ 800,000 = 4배

재귀 비중 = 3,200,000 ÷ (3,200,000 + 800,000) = 80.0%

→ 하드파싱이 딕셔너리 재귀 SQL 320만 콜을 낳았다(하드파싱의 부산물)

시간의 정체도 파싱 축입니다. 비재귀 Parse에 CPU 32.0초가 잡혀 있습니다.

Parse CPU 비중(비재귀) = 32.0 ÷ 39.0 × 100 ≈ 82.1% ← 계획 생성(하드파싱) CPU

Execute 비중 = 6.0 ÷ 39.0 × 100 ≈ 15.4% ← 실제 갱신은 소량

여기서 두 축을 분리해야 합니다.

축 A: 하드파싱(계획 생성·라이브러리 캐시 MISS·재귀 딕셔너리 콜) — 미스 99.0%, 재귀 4배 → 병목

축 B: 파싱 콜 횟수(소프트파싱·커서 미보유) — Parse:Execute=1:1이지만 여기선 소프트파싱이 아님

동적 SQL이 리터럴로 텍스트를 매번 바꾸니 커서가 공유되지 않는다. Parse 콜을 캐싱으로 붙잡아도(축 B 처방)

텍스트가 다른 커서는 재사용되지 않는다. 원인은 축 A(하드파싱)이므로 처방도 축 A라야 한다.

처방은 커서 홀드가 아니라 동적 SQL의 리터럴을 바인드 변수로 바꿔 정적(Static) SQL로 SQL 텍스트를 하나로 고정하는 것입니다. 정적 SQL은 바인드 변수를 자동으로 써 텍스트가 같아지므로 커서가 공유돼 하드파싱이 소프트파싱으로 바뀌고, 딕셔너리 재귀 콜(320만)과 Parse CPU(32초)가 함께 사라집니다. ③이 하드파싱 축(계획 생성·재귀 콜)을 정확히 지목했습니다.

선택지	판정	이유
①	×	인용한 Execute 640만 + Fetch 960만 = 1,600만도 재귀 18.0초도 Disk 0의 논리 읽기이며, 재귀 statement는 하드파싱이 계획을 만들며 부른 디렉터리 조회입니다. 원인이 아니라 결과이므로 버퍼 캐시를 키워도 하드파싱이 계속되는 한 재귀 콜 320만은 다시 발생합니다. 병목의 뿌리(하드파싱)가 아니라 부산물(재귀 논리 읽기)을 겨냥한 헛짚음입니다
②	×	Parse:Execute = 1:1과 Parse CPU 32.0초는 사실이지만, 이를 소프트파싱 쪽주로 읽은 것이 오류입니다. 라이브러리 캐시 미스가 99.0%라 이 파싱들은 소프트파싱이 아니라 하드파싱이고, 리터럴로 텍스트가 매번 달라 커서 홀드· session_cached_cursors 로 붙잡을 공유 커서 자체가 없습니다. 소프트파싱 빈도 축을 하드파싱 축으로 오인한 별개 축 혼동입니다
③	○	하드파싱율 396,000/400,000 = 99.0%로 계획 생성 축이 병목임을 짚고, 그 하드파싱이 재귀 디렉터리 콜 320만(비재귀의 4배)을 유발했음을 연결한 뒤, 동적 SQL의 리터럴을 바인드 변수(정적 SQL)로 바꿔 커서를 공유시키는 처방이 정확합니다
④	×	행당 Query 2·Current 1블록은 맞지만 Execute CPU 5.0초는 전체 39초의 15.4%뿐이고, 재귀 18.0초는 갱신을 뒷받침하는 비용이 아니라 하드파싱이 부른 디렉터리 조회입니다. 병목은 실행 축이 아니라 Parse CPU 32초(82.1%)의 하드파싱 축이므로, UPDATE 실행계획을 다시 짜도 파싱 지연은 그대로 남습니다

■ 이 문제의 핵심

- 하드파싱율 = 라이브러리 캐시 미스 ÷ Parse 콜. 여기서는 396,000/400,000 = 99.0%로, 동적 SQL이 리터럴을 붙여 텍스트가 매번 달라 커서가 공유되지 못해 사실상 매 실행이 하드파싱입니다.
 - 하드파싱은 재귀 Call을 낳습니다. 계획을 새로 만들 때마다 데이터 디렉터리를 조회하므로 재귀 statement가 320만 콜(비재귀의 4배)로 폭증했습니다 — 재귀 축은 하드파싱의 부산물입니다.
 - 하드파싱 축(계획 생성)과 소프트파싱 빈도 축(커서 미보유)은 별개입니다. Parse:Execute = 1:1이어도 미스율이 99.0%면 커서 캐시는 붙잡을 공유 커서가 없어 무력합니다.
 - 처방은 커서 홀드·버퍼 캐시·실행계획 튜닝이 아니라 동적 SQL의 리터럴 → 바인드 변수(정적 SQL)입니다. 정적 SQL은 바인드 변수를 자동으로 써 텍스트가 하나로 고정돼 커서가 공유됩니다.
 - Parse CPU 32초(82.1%)가 시간을 지배하고 Execute는 15.4%뿐이라, 병목은 실행이 아니라 파싱(계획 생성)에 있습니다.
- 한 줄 정리 하드파싱율이 99.0%(396,000/400,000)이고 재귀 디렉터리 콜이 320만(비재귀의 4배)으로 폭증한 것은 동적 SQL의 리터럴 때문에 커서가 공유되지 못해 매 실행이 하드파싱이기 때문이며, 처방은 커서 홀드·버퍼 캐시가 아니라 리터럴을 바인드 변수(정적 SQL)로 바꿔 하드파싱을 소프트파싱으로 돌리는 것이다.

문제 4. 정답 ①

'예상 계획을 뽑는 도구'와 '실제 수행된 계획을 보는 도구'의 경계를 흐리지 않고 갈라야 합니다.

예상 계획 (추정)	: EXPLAIN PLAN → PLAN_TABLE, DBMS_XPLAN.DISPLAY - SQL을 실행하지 않고 옵티마이저가 산출한 계획 - 바인드 값·데이터 분포에 따라 실제와 달라질 수 있음
실제 계획 (수행됨)	: DBMS_XPLAN.DISPLAY_CURSOR - 이미 실행되어 커서 캐시(라이브러리 캐시)에 올라온 계획을 조회
계획 안정성(고정)	: SQL Plan Baseline / Stored Outline - 계획 '선택'을 제약할 뿐, 파싱·최적화 과정을 없애지는 않음

EXPLAIN PLAN·DISPLAY로 보는 것은 옵티마이저가 실행 전에 추정한 예상 계획입니다. 바인드 변수 값이나 실제 데이터 분포에 따라 실행 시점의 계획이 달라질 수 있으므로, 실제로 무엇이 수행됐는지는 커서 캐시에 올라온 계획을 DISPLAY_CURSOR로 확인해야 정확합니다. ①은 두 도구의 경계를 바르게 나눴습니다.

②는 계획을 고정하는 것과 최적화를 생각하는 것을 뒤섞었습니다. Baseline·Outline은 옵티마이저가 고를 수 있는 계획을 제약할 뿐, 파싱과 계획 평가 과정 자체를 없애지 않으며 등록된 계획이 검증에서 탈락하면 다른 계획이 쓰이므로 "어긋날 여지가 없다"고 할 수도 없습니다. ③은 EXPLAIN PLAN이 SQL을 실제로 실행한다고 오해했습니다. EXPLAIN PLAN은 수행 없이 추정만 하므로 실제 처리 건수·소요 시간을 측정하지 않고, PLAN_TABLE에 담기는 값도 추정치이며 그 계획이 실제와 일치한다는 보장도 없습니다. ④는 DISPLAY_CURSOR가 커서 캐시에 올라온(=이미 실행된) 계획을 읽는 도구라는 경계를 무시했습니다. SQL_ID로 커서를 찾는 방식이라 실행되지 않아 커서가 없는 SQL은 애초에 조회 대상이 되지 않습니다.

선택지	판정	이유
①	○	DISPLAY로 보는 것은 실행 전 예상 계획이라 실제와 달라질 수 있고, 실제 수행된 계획은 커서 캐시를 읽는 DISPLAY_CURSOR로 확인해야 정확합니다
②	×	Baseline·Outline은 계획 '선택'을 제약할 뿐이라 파싱과 계획 평가는 그대로 일어나고, 등록 계획을 꺼내 쓴다고 해서 옵티마이징 비용이 사라지지도 예상 계획과 실제 계획의 괴리가 없어지지도 않습니다
③	×	EXPLAIN PLAN은 SQL을 실행하지 않고 추정만 하므로 실제 처리 건수도 소요 시간도 측정하지 않습니다. PLAN_TABLE에 적재되는 값은 추정치이고, 바인드 값이 달라지면 실행 시점 계획과 어긋날 수 있습니다
④	×	DISPLAY_CURSOR는 SQL_ID로 커서 캐시를 찾아 이미 실행된 계획을 읽는 도구입니다. 커서가 없는 SQL은 조회 대상 자체가 없으므로, 실행 전 예상 계획은 EXPLAIN PLAN·PLAN_TABLE 쪽으로 가야 합니다

■ 이 문제의 핵심

1. EXPLAIN PLAN·DISPLAY는 실행 전 예상 계획을 추정해 보여 줄 뿐, 실행하지 않습니다.
2. 실제 수행된 계획은 커서 캐시를 읽는 DISPLAY_CURSOR로 확인해야 정확합니다.
3. 예상 계획은 바인드 값·데이터 분포에 따라 실제 계획과 달라질 수 있습니다.
4. SQL Plan Baseline·Stored Outline은 계획 선택을 고정할 뿐 파싱·최적화 과정을 없애지 않습니다.

■ 한 줄 정리 DISPLAY는 실행 전 예상 계획, DISPLAY_CURSOR는 커서 캐시의 실제 계획을 보는 도구이며(①), 계획 고정 수단이 최적화를 생략한다거나 EXPLAIN PLAN이 SQL을 실행한다는 서술은 도구의 경계를 흐린 것입니다.

문제 5. 정답 ①

먼저 BCHR로 논리 대비 물리 읽기를 봅니다.

$$\begin{aligned} \text{BCHR} &= ((\text{Query} + \text{Current}) - \text{Disk}) / (\text{Query} + \text{Current}) \times 100 \\ &= (5,000,000 + 0 - 100,000) / 5,000,000 \times 100 \\ &= 4,900,000 / 5,000,000 \times 100 = 98.0\% \end{aligned}$$

$$\text{물리 읽기 비중} = 100,000 / 5,000,000 \times 100 = 2.0\%$$

캐시 적중이 98%로 높습니다. 논리 읽기 500만 중 실제 디스크에서 올라온 것은 10만(2%)뿐입니다. 그런데도 느립니다 — 이때는 물리 I/O가 아니라 논리 읽기 자체의 양을 의심해야 합니다. 시간의 정체를 봅니다.

$$\begin{aligned} \text{CPU 비중} &= 32.000 / 40.000 \times 100 = 80.0\% \quad \rightarrow \text{CPU가 시간을 지배} \\ \text{대기 시간} &= 40.000 - 32.000 = 8.0\text{초} \quad (20\%) \quad \leftarrow \text{그중 물리 읽기 대기} \end{aligned}$$

응답의 80%가 CPU입니다. 물리 읽기가 작는데 CPU가 큰 것은, 캐시에 있는 블록을 반복해서 논리적으로 읽느라(buffer get) CPU를 태운다는 신호입니다. 어디서 났는지 Row Source로 찾되, cr(논리)과 pr(물리)을 분리합니다.

pw(temp spill) = 모든 노드 0 → spill 없음

논리 읽기(cr) 정체 = NL 조인의 부이관측값 반복 방문 = 인덱스 재탐색 3,200,000 + 테이블 랜덤 self 1,799,950
(테이블 self = TABLE ACCESS BY INDEX ROWID 4,999,950 - 자식 INDEX 3,200,000 = 1,799,950)

→ NL 조인이 400건 × (인덱스+테이블) 방문을 반복해 200만 행마다 블록을 논리적으로 읽음 → cr 500만

물리 읽기(pr) = 100,000 (부이관측값), 그 노드 time의 대부분은 물리 대기가 아니라 논리 읽기 CPU

pw=0이라 spill은 없고, **pr**은 10만으로 작습니다. 정체는 NL 조인이 **부이관측값**을 200만 번 방문하며 인덱스 재탐색(cr 3,200,000)과 테이블 랜덤 액세스(self 1,799,950)로 쌓은 논리 읽기 500만이고, 그 대부분이 캐시 적중이라 디스크가 아닌 CPU를 태웁니다.

BCHR 98% 높음 → 물리 디스크는 병목이 아님(Disk 2%)

CPU 80% 지배 → 논리 읽기(buffer get) 500만이 CPU를 소진 → 논리 I/O 바운드

따라서 처방은 물리 읽기가 아니라 논리 읽기 횟수 자체를 줄이는 것 — NL 조인을 해서 조인으로 바꾸거나 액세스 경로를 개선해 **부이관측값** 방문 횟수를 낮추는 것입니다. ①이 BCHR과 병목 지목을 함께 옳게 했습니다.

선택지	판정	이유
①	○	BCHR (5,000,000-100,000)/5,000,000 = 98.0%로 캐시 적중이 높아 물리 읽기가 2%뿐임을 짚고, CPU 80%·pw=0·cr 500만이 부이관측값 반복 방문에 쌓였음을 연결해 병목을 논리 읽기(buffer get)로 지목하고 논리 읽기 축소를 처방한 것이 정확합니다
②	×	pr이 부이관측값 노드에 물린 것은 사실이나 그 100,000은 같은 노드 cr 4,999,950의 2%뿐이고 BCHR은 98%입니다. 그 노드의 time 32,000,000us도 대부분 물리 대기가 아니라 논리 읽기 CPU이며, 버퍼 캐시를 키워 2%를 없애도 CPU 80%를 만드는 논리 읽기 500만은 그대로이므로 병목(논리 I/O)을 물리 I/O로 오귀속한 것입니다
③	×	Elapsed-CPU = 8초와 인덱스 pr=0은 맞지만 8초는 전체의 20%뿐이고 시간의 80%는 CPU입니다. 대기가 부이관측값 노드에 물렸다는 사실도 그 노드 time 32초의 정체가 cr 4,999,950이라는 점을 뒤집지 못하므로, DOP를 올리자는 것은 논리 읽기 병목을 물리 대기로 오귀속한 진단입니다
④	×	모든 노드가 pw=0이라 temp spill은 없고, Fetch 2001회는 배열 인출 왕복(2,000,000행 기준)이지 temp 되읽기가 아닙니다. NL 조인에는 대량 해시·정렬 영역이 개입하지 않으며 200만 행은 조인 결과일 뿐이므로, PGA를 키워도 40초는 그대로입니다 — 논리 읽기 CPU를 집계 spill로 오귀속한 것입니다

■ 이 문제의 핵심

1. $BCHR = ((Query + Current) - Disk) / (Query + Current) \times 100$. 여기서는 $(5,000,000 - 100,000) / 5,000,000 = 98.0\%$ 로 캐시 적중이 높아, 논리 읽기의 2%만 디스크에서 올라왔습니다.
2. BCHR이 높음에도 느리면 논리 읽기 양을 의심합니다. 물리 I/O가 작는데 CPU가 80%면, 캐시에 있는 블록을 반복 논리 읽기(buffer get)하며 CPU를 태우는 논리 I/O 바운드입니다.
3. cr과 pr·pw를 분리해야 합니다. cr 500만은 NL 조인이 **부이관측값**을 200만 번 방문해 쌓은 것이고, pr은 10만·pw는 0입니다.
4. 높은 BCHR은 항상 좋은 신호가 아닙니다. 불필요한 논리 읽기가 많아도 캐시에서 나오면 BCHR은 높게 뜨므로, 병목은 물리가 아니라 논리 읽기 횟수일 수 있습니다.
5. 처방은 논리 읽기 축소(NL→해시 조인 전환·액세스 경로 개선)입니다. 버퍼 캐시·DOP·PGA는 CPU를 태우는 논리 I/O를 근거부터 잘못 겨냥합니다.

- 한 줄 정리 BCHR 98%로 캐시 적중이 높아 물리 읽기가 2%뿐이고 pw=0이라 spill도 없는데 CPU가 80%인 것은 NL 조인이 **부이관측값**을 200만 번 방문해 논리 읽기 500만을 쌓았기 때문이므로 병목은 논리 I/O(buffer get)이고, 이를 물리 읽기·DOP·집계 spill로 돌리면 원인을 오귀속하게 된다.

문제 6. 정답 ④

플랜의 두 노드에서 나온 Card 값을 나란히 놓습니다.

INDEX RANGE SCAN 점검_IX Card 7,200 ← 인덱스에서 훑고 남은 건수
TABLE ACCESS BY INDEX ROWID 안전점검 Card 1,900 ← 테이블까지 거친 뒤 남은 건수

점검_IX는 (관할지사코드, 점검일자) 순서입니다. WHERE에는 **관할지사코드 = 'D07'**(등치)와 **점검일자** 범위가 있으므로, 선두 칼럼 등치 + 두 번째 칼럼 범위까지가 인덱스로 연속해서 훑을 수 있는 구간입니다. 그 결과가 INDEX RANGE SCAN의 7,200건입니다.

판정결과는 **점검_IX**의 구성 칼럼이 아닙니다. 인덱스에 값이 없으니 인덱스 단계에서는 걸러낼 수 없고, ROWID로 테이블 블록을 읽어 온 TABLE ACCESS BY INDEX ROWID 단계에서 필터로 처리됩니다. 그래서 7,200건이 이 단계를 거치며 1,900건으로 줄어듭니다.

7,200 (인덱스 액세스: 관할지사코드=, 점검일자 범위)
└─ 테이블 필터(판정결과='부적합') 적용 → 1,900 (최종)

즉 ④가 이 구조를 정확히 서술합니다. 판정결과까지 인덱스 액세스 조건으로 끌어올린 ①, 인덱스 반환 건수를 최종 건수로 본 ②, 선두 칼럼 등치까지 필터로 밀어 넣은 ③은 이 Card 흐름과 어긋납니다.

선택지	판정	이유
①	×	판정결과는 점검_IX 구성 칼럼이 아니므로 인덱스 액세스 조건이 될 수 없습니다. 테이블 필터인 판정결과를 인덱스 액세스 조건 집합에 끼워 넣은 진술로, 세 조건이 Cost 22의 인덱스 스캔에서 함께 걸러졌다면 그 단계 Card가 이미 1,900이었을 텐데 실제로는 7,200이고 1,900은 상위 테이블 노드에서야 나옵니다
②	×	Card 7,200은 인덱스 단계 추정 행 수일 뿐이고, 판정결과 필터를 거친 최종 Card는 1,900입니다. Card 열은 블록 수가 아니라 행 수를 뜻하며(같은 노드 Bytes 79,800과 짝을 이룹니다), 테이블 액세스 단계에서 7,200→1,900의 건수 감소가 분명히 일어납니다
③	×	두 번째 칼럼이 범위라는 성질을 선두 칼럼에까지 확대한 진술입니다. 선두 칼럼 등치는 그 자체로 액세스 조건이고, 범위는 그 다음 칼럼부터 적용됩니다. 등치까지 필터였다면 940만 건 전체를 훑어야 해 Cost 22·Card 7,200으로 끝나는 INDEX RANGE SCAN이 성립하지 않습니다
④	○	관할지사코드(등치)·점검일자(범위)는 인덱스 액세스 조건, 판정결과는 인덱스에 없어 TABLE ACCESS 단계 필터 — Card가 7,200→1,900으로 줄어드는 흐름과 일치합니다

■ 이 문제의 핵심

1. 결합 인덱스는 선두 칼럼 등치 + 다음 칼럼 범위까지가 액세스 조건. 여기서는 관할지사코드(=)·점검일자(범위)가 INDEX RANGE SCAN 구간을 만듭니다(Card 7,200).
2. 인덱스에 없는 칼럼은 테이블 필터. 판정결과는 ROWID로 테이블을 읽은 뒤 TABLE ACCESS BY INDEX ROWID 단계에서 걸러집니다.
3. 두 노드의 Card 차이(7,200 vs 1,900)가 곧 테이블 필터의 효과입니다. 인덱스 반환 건수 ≠ 최종 건수.
4. 테이블 필터 조건을 인덱스 액세스 조건 집합에 끼워 넣지 않습니다. 인덱스에 없는 판정결과는 스캔 범위를 정하는 데 관여하지 못합니다.

■ 한 줄 정리 선두 칼럼 등치(관할지사코드)와 다음 칼럼 범위(점검일자)는 인덱스 액세스 조건이 되고, 인덱스에 없는 판정결과는 테이블 액세스 단계 필터가 되어 Card가 7,200에서 1,900으로 줄어듭니다.

문제 7. 정답 ④

Row Source의 cr는 누적이므로, 테이블 액세스 노드의 246,250에서 자식 인덱스 1,250을 빼면 테이블 자체 블록 I/O가 나옵니다. 이를 ROWID 수로 나눠 클러스터링 팩터를 가늠합니다.

테이블 자체 블록 I/O = 246,250 - 1,250 = 245,000

액세스 ROWID(행) 수 = 250,000

CF 추정 = 245,000 / 250,000 ≈ 0.98 블록/행

→ 행마다 거의 다른 블록 → 클러스터링 팩터 최악(≈ 행 수에 근접)

CF가 나쁜 인덱스로 뽑은 각 행이 서로 다른 테이블 블록에 흩어져 있어, 테이블 Random 액세스가 한 행당 블록 하나씩을 단일 블록으로 읽습니다. 어디에 시간이 쏟렸는지 보니다.

테이블 액세스 cr 비중 = 245,000 / 246,250 × 100 ≈ 99.5% ← 논리 읽기의 대부분

인덱스 스캔 cr 비중 = 1,250 / 246,250 × 100 ≈ 0.5% ← 극소

테이블 액세스 time = 29.6초 (인덱스 스캔 time은 0.35초)

이제 손익분기점입니다. 테이블은 약 40만 블록인데, 인덱스 경유로 읽는 테이블 블록이 245,000입니다.

인덱스 경유 테이블 블록 = 245,000

테이블 총 블록 = 40,000,000 / 100 = 400,000

비율 = 245,000 / 400,000 = 61.25%

→ 테이블의 3분의 2 가까이를 '단일 블록 랜덤 읽기'라는 느린 경로로 훑음

→ 다중 블록 Full Scan 손익분기점을 이미 넘음

인덱스 스캔(cr 1,250·0.35초)은 문제가 아닙니다. 정체는 CF 최악으로 부풀어 오른 테이블 Random 액세스

245,000블록이며, 이는 필요한 컬럼을 인덱스에 더한 커버링 인덱스로 테이블 액세스 자체를 없애거나 Full Scan으로 전환할 때 사라집니다. 따라서 ④가 원인을 정확히 지목했습니다.

선택지	판정	이유
①	×	인용한 cr 1,250 · pr 0 · time 350,000us가 바로 이 스캔이 무죄라는 증거입니다. 인덱스는 전체 cr의 0.5% · 0.35초만 쓰므로 리빌드로 리프를 다져도 상위 노드의 time 29.6초와 245,000블록 테이블 Random 액세스는 그대로여서, 병목을 인덱스로 오귀속한 것입니다
②	×	Parse · Execute가 0에 가깝고 시간이 Fetch에 몰린 것은 맞지만 Fetch 501회는 배열 크기 500(=250,000/500)의 결과일 뿐이고, 29.996초는 왕복 대기가 아니라 Row Source가 보여 주듯 테이블 노드 time 29.6초에서 났습니다. 배열 크기를 키워도 읽어야 할 245,000블록은 줄지 않습니다
③	×	pr 50,000이 테이블 노드에 몰린 것은 맞지만, 버퍼 캐시는 블록을 어디서 가져오는지만 바꿀 뿐 방문 횟수를 줄이지 못합니다. 50,000을 캐시 적중으로 바꿔도 CF ≈ 0.98이 만든 245,000블록의 논리 랜덤 액세스는 그대로 남아 손익분기점(61.25%)은 그대로이므로, 26.5초의 원인을 물리 읽기로 오귀속하고 그 효과까지 부풀린 것입니다
④	○	테이블 자체 cr 245,000(=246,250-1,250)이 ROWID 250,000과 거의 같아 CF ≈ 0.98로 최악이고, 이 245,000이 cr의 99.5% · 29.6초를 차지하며 40만 블록의 61.25%를 랜덤 읽기로 훑어 손익분기점을 넘었다는 지목과 커버링 인덱스 처방이 정확합니다

■ 이 문제의 핵심

1. Row Source cr는 누적입니다. 테이블 자체 블록 I/O = 상위(246,250) - 자식 인덱스(1,250) = 245,000. 이걸 ROWID 수로 나눠 CF를 가늠합니다(245,000/250,000 ≈ 0.98 블록/행).
2. CF가 최악이면 테이블 Random 액세스가 병목입니다. 245,000이 cr의 99.5% · time 29.6초를 차지하고, 인덱스 스캔(1,250 · 0.35초)은 무죄입니다.
3. 손익분기점: 인덱스 경유 테이블 블록 245,000이 총 40만 블록의 61.25%를 단일 블록 랜덤으로 훑으면 다중 블록 Full Scan보다 불리합니다.
4. 처방은 인덱스 리빌드 · 배열 크기 · 버퍼 캐시가 아니라 테이블 액세스 자체 제거(커버링 인덱스) 또는 Full Scan 전환입니다.

- 한 줄 정리 테이블 자체 cr 245,000이 ROWID 250,000과 거의 같아 CF가 최악이고, 이 245,000블록의 랜덤 액세스가 cr의 99.5% · 29.6초를 차지하며 손익분기점(61.25%)을 넘었으므로 처방은 인덱스 · 캐시가 아니라 커버링 인덱스로 테이블 액세스를 없애는 것이다.

문제 8. 정답 ②

Range Scan의 '스캔 범위'를 정하는 것이 인덱스 액세스 조건인지, 인덱스 밖 필터인지를 갈라야 합니다.

스캔 범위(리프 시작점~종료점)를 정하는 것

= 인덱스 선두 컬럼에 걸린 '액세스 조건' ← 시작점 · 종료점을 줍힘

인덱스에 없는 '필터 조건'

= 스캔 범위는 그대로, 훑는 도중 행을 걸러 낼 뿐 ← 리프 구간 폭을 줄이지 못함

Index Range Scan에서 실제로 훑는 리프 구간의 폭은 인덱스 선두 컬럼에 걸린 액세스 조건이 정합니다. 액세스 조건이 시작점과 종료점을 좁히고, 인덱스에 없는 일반 필터 조건은 스캔 범위를 줄이지 못한 채 훑는 도중 행을 걸러 내지만 합니다.

②는 이 관계를 정반대로 뒤집었습니다. "인덱스 선두 컬럼 조건과 무관하게 인덱스 밖 필터가 많을수록 범위가 좁아지고 그 필터가 시작 · 종료점을 정한다"고 했지만, 필터 조건은 시작 · 종료점을 정하지 못하며 스캔 범위 자체를 줄이지도 못합니다. 시작 · 종료점을 정하는 것은 인덱스 액세스 조건입니다. 필터가 하는 일과 액세스 조건이 하는 일을 서로 맞바꾼 서술입니다.

① · ③ · ④는 옳습니다. 유일 인덱스 등치로 한 건만 찾고 끝내는 Unique Scan의 성립 조건(①), 스캔 범위가 선두 컬럼 액세스 조건으로 정해지고 나머지는 필터로만 걸린다는 원리(③), 선두 컬럼 범위 조건이 폭을 넓히고 뒤쪽 컬럼이 시작점을 못 좁힌다는 점(④)이 모두 인덱스 스캔 범위의 실제 동작에 부합합니다.

선택지	판정	이유
①	○	유일 인덱스 등치 조건은 최대 한 건이라, 리프 엔트리 하나를 찾으려면 더 읽을 것이 없어 곧바로 끝나는 Unique Scan이 성립합니다
②	×	스캔 시작 · 종료점을 정해 범위를 좁히는 것은 인덱스 선두 컬럼의 액세스 조건이며, 인덱스 밖 필터는 범위를 줄이지 못하고 훑는 중 걸러 낼 뿐입니다. 필터와 액세스 조건의 역할을 정반대로 맞바꾼 서술입니다
③	○	스캔 시작 · 종료점은 선두 컬럼 액세스 조건이 정하고, 그에 못 걸린 조건은 범위를 줄이지 못한 채 필터로만 걸러집니다
④	○	선두 컬럼에 범위 조건이 걸리면 시작 · 종료 폭이 넓어져 훑는 리프가 늘고, 뒤쪽 컬럼 조건은 시작점을 더 좁히지 못합니다

■ 이 문제의 핵심

1. Index Unique Scan은 유일 인덱스 등치로 최대 한 건일 때 성립해 하나 찾고 곧바로 끝납니다.
2. Index Range Scan의 스캔 범위(시작·종료점)는 인덱스 선두 컬럼 액세스 조건이 정합니다.
3. 인덱스에 없는 필터 조건은 스캔 범위를 줄이지 못하고, 훑는 도중 행을 걸러 낼 뿐입니다.
4. 선두 컬럼 범위 조건은 폭을 넓히고, 뒤쪽 컬럼 조건은 시작점을 더 좁히지 못합니다.

■ 한 줄 정리 Range Scan의 스캔 범위를 정하는 것은 인덱스 선두 컬럼의 액세스 조건이고 필터는 범위를 줄이지 못하는데, "인덱스 밖 필터가 시작·종료점을 정한다"는 ②는 두 역할을 정반대로 맞바꾼 서술입니다.

문제 9. 정답 ②

비트맵 인덱스의 DML 부하가 '세그먼트 크기' 하나로 환원되는지, 아니면 더 근본적인 요인이 있는지를 갈라야 합니다.

비트맵 인덱스의 DML 부하 = 여러 요인

└─ 요인 A: 락 단위 → 한 비트맵 조각이 여러 행을 덮어, 한 행 변경이 그 조각 전체를 잠금
└─ → 같은 비트맵을 건드리는 동시 DML이 서로 대기(직렬화) ← OLTP에 치명적
└─ 요인 B: 세그먼트 크기·재구성 비용

- ② : DML 부하를 요인 B(크기) '하나'로 환원 → 크기만 줄이면 해소된다고 결론
→ 진짜 걸림돌인 요인 A(락 직렬화)를 통째로 누락 ← 부분을 전체로 끼운 오류

비트맵 인덱스가 DML에 불리한 근본 원인은 락 단위입니다. 비트맵 한 조각이 여러 행을 덮기 때문에, 한 행만 바뀌도 그 조각에 락이 걸려 같은 비트맵을 갱신하려는 다른 트랜잭션이 대기하게 됩니다. 이 직렬화가 동시 DML이 잦은 OLTP에서 비트맵을 쓰기 어렵게 만드는 진짜 이유입니다.

②는 이 여러 요인 가운데 하나(세그먼트 크기)만 떼어 내 그것이 DML 부하의 전부인 것처럼 서술했습니다. "크기 때문일 뿐이니 세그먼트만 작게 하면 해소된다"는 결론은, 카디널리티를 낮춰 크기를 줄이더라도 락 직렬화는 그대로라는 사실을 누락한 부분→전체 오류입니다. 오히려 낮은 카디널리티는 한 비트맵 조각이 더 많은 행을 덮게 만들어 락 경합을 키우는 방향입니다.

①·③·④는 옳습니다. 낮은 카디널리티·비트 연산·OLAP 적합성(①), 비트맵 락 단위로 인한 동시 DML 대기(③), 카디널리티와 DML 빈도를 함께 따지는 설계 원칙(④)이 모두 비트맵 인덱스의 실제 성질에 부합합니다.

선택지	판정	이유
①	○	비트맵 인덱스는 낮은 카디널리티 컬럼에서 값마다 비트맵을 만들어 AND·OR 비트 연산으로 집계하기 좋아, 갱신이 드문 OLAP·DW에 적합합니다
②	×	비트맵의 DML 부하는 세그먼트 크기 하나가 아니라 락 단위(직렬화)가 근본 원인이라, 크기만 줄여도 해소되지 않습니다. 부분(크기)을 전체로 끼운 서술입니다
③	○	한 비트맵 조각이 여러 행을 덮어, 한 행 변경도 그 조각에 락을 걸어 같은 비트맵을 갱신하려는 다른 트랜잭션을 대기시킵니다
④	○	인덱스 설계는 조회 패턴·카디널리티·DML 빈도를 함께 따져야 하며, DW에서는 비트맵이 OLTP에서는 B*Tree가 유리할 수 있습니다

■ 이 문제의 핵심

1. 비트맵 인덱스는 낮은 카디널리티 컬럼에서 비트 연산으로 집계하기 좋아 OLAP·DW에 맞습니다.
2. 비트맵의 DML 부하 근본 원인은 락 단위(한 조각이 여러 행 → 조각 전체 잠금 → 직렬화)입니다.
3. 세그먼트 크기는 DML 부하의 한 요인일 뿐, 크기를 줄여도 락 직렬화는 남습니다.
4. 인덱스 설계는 카디널리티와 DML 빈도를 함께 따져야 합니다.

■ 한 줄 정리 비트맵 인덱스의 DML 부하는 락 단위(직렬화)가 근본 원인인데, 그것을 세그먼트 크기 하나로 환원해 "크기만 줄이면 OLTP에서도 쓸 수 있다"고 한 ②는 부분을 전체로 끼운 서술입니다.

문제 10. 정답 ①

플랜에 어떤 노드가 있고 없는지를 그대로 읽습니다.

Id 1 COUNT STOPKEY ROWNUM<=20 도달 시 하위에 정지 신호
Id 2 VIEW
Id 3 TABLE ACCESS BY INDEX ROWID 상표출원 인덱스가 준 순서대로 20건만 테이블 접근
Id 4 INDEX FULL SCAN DESCENDING 출원일시_IX 최신순(역방향)으로 앞에서부터, 20건 차면 정지

이 플랜에는 SORT ORDER BY 노드가 없습니다. **출원일시_IX**를 역방향(DESCENDING)으로 스캔하므로 스캔 순서가 곧 최신순이고, **COUNT STOPKEY**(Id 1)가 ROWNUM 한도 20에 도달하는 순간 하위 스캔을 멈춥니다. 즉 인덱스를 앞에서 20건만 읽고 그 20건에 대해서만 테이블을 접근한 뒤 끝나는 부분범위처리입니다.

①은 이 동작을 "전체 정렬 + 테이블 전체 읽기"로 뒤바꿔 서술합니다. 전체 정렬이라면 SORT ORDER BY 노드가 있어야 하는데 없고, 전체 읽기라면 STOPKEY가 무의미해집니다. 실제 플랜은 정렬 노드 없이 인덱스 순서를 쓰고 20건에서 스캔을 끊으므로, ①은 STOPKEY 기반 부분범위처리를 정반대의 전량 처리로 오독한 값스왑 진술입니다. 따라서 "가장 적절하지 않은 것"은 ①입니다.

- ① 전체 정렬 + 테이블 전체 읽기 : SORT 노드 있어야 · STOPKEY 무의미 → 플랜엔 SORT 없음, STOPKEY 있음 (틀림)
- ② COUNT STOPKEY로 20건에서 정지 : Id 1과 일치 ○
- ③ 인덱스가 순서 제공, SORT 없음 : Id 4 INDEX FULL SCAN DESCENDING 근거 ○
- ④ 앞 20건까지만 접근하는 부분범위 : STOPKEY + 내림차순 인덱스 동작과 일치 ○

②③④는 STOPKEY·내림차순 인덱스·부분범위처리의 동작을 정확히 서술합니다. ①만 이를 전량 정렬·전량 스캔으로 뒤바꿨습니다.

선택지	판정	이유
①	×	플랜에 SORT ORDER BY 노드가 없고 Id 1은 COUNT STOPKEY 입니다. 6,400만 건을 전부 정렬하지도, 테이블을 끝까지 읽지도 않습니다 — 20건에서 멈추는 부분범위처리를 전량 정렬·전량 스캔으로 뒤바꾼 값스왑 진술입니다
②	○	Id 1 COUNT STOPKEY 는 ROWNUM 한도 20 도달 시 하위 노드에 정지 신호를 보내므로, 20건을 채우면 그 아래 스캔이 멈추는 STOPKEY 동작과 일치합니다
③	○	Id 4가 INDEX FULL SCAN DESCENDING 출원일시_IX 이고 별도 SORT 노드가 없으므로, 인덱스를 역방향 스캔해 최신순 순서를 그대로 쓴다는 서술이 맞습니다
④	○	내림차순 인덱스를 앞에서부터 훑어 20건이 차는 지점까지만 접근하는 것이 STOPKEY 기반 부분범위처리이며, 나머지 방대한 구간은 접근하지 않습니다

▪ 이 문제의 핵심

- Top-N 부분범위처리의 정지는 **COUNT STOPKEY**가 담당합니다. ROWNUM 한도에 도달하면 하위 스캔을 끊어 앞 N건만 읽습니다.
- 내림차순 인덱스가 정렬 순서를 제공하면 SORT ORDER BY 노드가 사라집니다. 이 플랜엔 정렬 노드가 없어 인덱스 순서를 그대로 최신순으로 씁니다.
- STOPKEY + 정렬된 인덱스 = 전량 처리 회피. 6,400만 건을 정렬하거나 끝까지 읽지 않고, 20건에서 멈춥니다.
- STOPKEY 플랜을 전체 정렬·전체 스캔으로 읽으면 정반대입니다. SORT 노드 유무와 STOPKEY 유무가 그 구分的 근거입니다.

▪ 한 줄 정리 Id 1 **COUNT STOPKEY**와 역방향 스캔 **출원일시_IX**(SORT 노드 없음)로 앞 20건만 읽고 멈추는 부분범위처리이므로, 이를 6,400만 건 전량 정렬·전량 스캔으로 서술한 ①이 가장 적절하지 않다.

문제 11. 정답 ③

해시 조인에서 성능의 갈림길은 어느 입력을 Build로 삼아 해시 테이블을 만드느냐이고, 이는 조인 순서·해시 영역 크기와 하나로 엮여 있습니다.

- Build Input 선택 : 옵티마이저가 '더 작다고 추정'한 집합 → 해시 테이블 구축 대상
- 조인 순서(먼저 처리할 쪽)와 맞물려 결정됨
 - Build가 Hash Area(PGA)에 다 담기면 → In-Memory Hash Join, Temp 경유 없음
 - Build가 Hash Area에 다 담기지 못하면 → 두 입력을 분할해 Temp 경유(Grace/multi-pass)
- ⇒ 작은 집합을 Build로 잡을수록 담길 확률이 커져 비용에 유리

③은 이 골격을 정확히 짚습니다. Build Input은 텍스트 순서가 아니라 추정 크기(비용)로 정해지고, 그 선택이 조인 순서 결정과 함께 이뤄집니다. Build가 해시 영역에 담기면 추가 I/O 없이 조인이 끝나므로, 작은 집합을 Build로 두는 것이 담길 가능성을 높여 유리합니다.

나머지는 키워드에서 곧장 결론으로 건너뛴 반사적 연상입니다.

- ①: 'FROM 절 앞=먼저=Build'라는 반사입니다. Build/Probe 선택도 조인 순서도 SQL 텍스트 순서가 아니라 옵티마이저의 크기 추정과 비용으로 정해지므로, 큰 테이블을 뒤에 적는다고 Build가 작아지거나 해시 영역 적재 확률이 올라가지 않습니다.
- ②: '해시 영역에 못 담기면 다른 조인으로 바뀐다'는 반사입니다. Build가 넘치면 두 입력을 같은 기준으로 분할해 Temp를 경유하는 Grace Hash로 진행하므로 Temp 내려쓰기가 없어지기는커녕 바로 그때 생기며, 실행 중에 소트 머지 조인으로 갈아타는 것도 아닙니다.

④: '대량 집합=해시니까 무조건 싸다'는 반사입니다. 해시 조인 비용은 Build가 해시 영역에 담기는지, Temp로 넘쳐 분할되는지, 인덱스가 있는지에 따라 달라지므로 '무관하게'가 성립하지 않고, 크기만으로 NL보다 낮다고 단정할 수 없습니다.

선택지	판정	이유
①	×	Build/Probe도 조인 순서도 옵티마이저의 크기 추정과 비용으로 정해지며 FROM 절 텍스트 순서로 고정되지 않습니다. '앞에 적은 것=Build'는 반사적 연상이고, 뒤로 미뤄 적는다고 해시 영역 적재 확률이 올라가지도 않습니다
②	×	Build가 해시 영역에 못 담기면 두 입력을 분할해 Temp를 경유하는 Grace Hash로 진행합니다. Temp 내려쓰기가 없어지는 것이 아니라 오히려 그때 생기며, 실행 중 소트 머지 조인으로 자동 전환하지도 않습니다
③	○	옵티마이저는 작다고 추정한 집합을 Build로 골라 조인 순서와 함께 결정하며, Build가 해시 영역에 담기면 Temp 경유 없이 끝나 작은 집합을 Build로 잡는 편이 유리합니다
④	×	해시 조인 비용은 Build의 해시 영역 적재·Temp 분할 경유·인덱스 유무에 좌우되므로 '무관하게'가 성립하지 않고, 어느 쪽을 Build로 잡느냐도 비용을 가릅니다. 대량이라는 이유만으로 NL보다 낮다고 볼 수 없습니다

■ 이 문제의 핵심

1. Build Input은 크기 추정으로 정해진다. 텍스트 순서가 아니라 옵티마이저가 더 작다고 본 집합이 해시 테이블 대상이 되며, 이는 조인 순서와 맞물립니다.
2. 해시 영역 적재 여부가 비용을 가른다. Build가 다 담기면 Temp 경유가 없고, 넘치면 분할해 Temp를 거칩니다 — 그래서 작은 집합을 Build로 두는 편이 유리합니다.
3. 넘쳐도 조인 방식이 바뀌지는 않는다. Grace Hash로 분할 처리할 뿐 실행 중 소트 머지 조인으로 갈아타지 않습니다.
4. 크기만으로 비용 우위가 정해지지 않는다. 해시 영역 적재·Temp·인덱스에 따라 NL과의 우열이 달라집니다.

■ 한 줄 정리 해시 조인의 Build Input은 옵티마이저가 작다고 추정한 집합으로 조인 순서와 함께 정해지고 해시 영역에 담기면 Temp 없이 끝나므로 작은 집합을 Build로 잡는 편이 유리한데, "앞에 적은 게 Build"·"못 담기면 소트 머지로 전환"·"대량이면 무조건 해시가 싸다"는 키워드에서 건너뛴 반사적 연상입니다.

문제 12. 정답 ②

스칼라 서브쿼리 캐싱은 그 서브쿼리를 메인 행마다 실제로 수행할 때 같은 입력값의 재수행을 생략하는 최적화입니다. 그런데 옵티마이저가 스칼라 서브쿼리를 조인으로 변환하면, 서브쿼리는 더 이상 행마다 수행되는 스칼라 서브쿼리가 아니라 집합 기반 조인으로 처리됩니다. 즉 조인 변환과 스칼라 서브쿼리 캐싱은 같은 실행에 함께 작동하는 것이 아니라 서로 갈리는 처리 방식입니다.

경로 A) 변환 안 함 : 스칼라 서브쿼리를 행마다 수행 → 입력값 캐싱(같은 값 재수행 생략)이 작동
경로 B) 조인 변환 : 서브쿼리를 아우터 조인으로 풀어 집합 처리 → 행별 수행 자체가 없어 캐싱 대상 아님
⇒ 둘은 맥일 관계에 가깝다 (동시에 겹쳐 작동하지 않음)

②는 '스칼라 서브쿼리=캐싱'이라는 반사로, 조인으로 바뀐 뒤에도 캐싱이 그대로 함께 걸린다고 본 것입니다. 조인 변환이 일어나면 행별 반복 수행이 사라져 캐싱이 겨냥하던 재수행 자체가 없어지므로, 캐싱과 조인 변환이 '동시에 이중으로' 효과를 낸다는 진술은 성립하지 않습니다. 그래서 부적절한 것은 ②입니다.

①③④는 모두 옳습니다.

①: 스칼라 서브쿼리는 조인(언네스팅)으로 변환될 수 있고, 변환되면 행별 반복 수행이 집합 기반 조인으로 바뀝니다.

③: 메인 행마다 1건 보장이 필요하므로 아우터 조인 형태가 되고, 2건 이상 반환 시 오류가 나는 1건 보장 의미는 변환 뒤에도 유지됩니다.

④: 변환은 비용 기반 선택이라, 캐싱이 잘 듣는 상황(구별 값 적고 반복 많음)에서는 변환하지 않고 캐싱으로 두는 편이 유리할 수 있습니다.

선택지	판정	이유
①	○	스칼라 서브쿼리는 조인으로 언네스팅될 수 있고, 변환되면 행별 반복 수행이 집합 기반 조인으로 처리됩니다
②	×	조인으로 변환되면 행별 수행이 사라져 캐싱이 겨냥하던 재수행 자체가 없어집니다. 캐싱과 조인 변환은 겹쳐 작동하는 것이 아니라 갈리는 방식이라 '이중으로 준다'는 반사적 연상입니다
③	○	행마다 1건 보장 때문에 아우터 조인 형태가 되며, 2건 이상 반환 시 오류가 나는 1건 보장 의미는 변환 뒤에도 유지됩니다
④	○	변환은 비용 기반이라 캐싱이 잘 듣는 상황에서는 변환 대신 스칼라 서브쿼리 캐싱으로 두는 편이 유리할 수 있습니다

▪ 이 문제의 핵심

1. 스칼라 서브쿼리는 조인으로 변환될 수 있다. 언네스팅되면 행별 반복 수행이 집합 기반 조인(주로 아우터 조인)으로 바뀝니다.
2. 캐싱과 조인 변환은 택일 관계에 가깝다. 조인으로 바뀌면 행별 수행이 사라져 입력값 캐싱이 겨냥하던 재수행 자체가 없어집니다 — 겹쳐서 이중으로 작동하지 않습니다.
3. 1건 보장 의미는 변환 뒤에도 유지된다. 아우터 조인 형태가 되고, 한 행에 2건 이상 매칭되면 오류가 나는 성질이 이어집니다.
4. 변환은 비용 기반 선택이다. 캐싱이 잘 듣는 상황에서는 변환하지 않는 편이 유리할 수 있습니다.

▪ 한 줄 정리 스칼라 서브쿼리가 조인으로 변환되면 행별 수행이 사라져 캐싱 대상 자체가 없어지므로 둘은 택일에 가까운데, "조인 변환과 캐싱이 동시에 이중으로 작동한다"는 ②는 '스칼라 서브쿼리=캐싱' 반사에 기댄 진술이라 부적절합니다.

문제 13. 정답 ④

결과 캐시 항목의 식별키는 SQL 텍스트만이 아니라 바인드 변수 값(그리고 NLS 등 일부 세션 파라미터)을 포함합니다. 텍스트가 같아도 :b1 값이 다르면 결과가 달라지므로, 오라클은 바인드 값별로 별도의 캐시 항목을 만들어 둡니다. '국보' 실행의 합계를 '보물' 실행이 재사용할 수는 없습니다 — 애초에 다른 결과이기 때문입니다.

④는 "텍스트가 같으면 바인드 값이 달라도 같은 캐시 결과를 재사용한다"는 정규화 가정의 오류입니다. 이는 결과 캐시가 텍스트를 정규화해 그 하나의 키로만 저장한다고 착각한 것으로, 실제로는 (텍스트 + 바인드 값) 조합마다 캐시가 걸립니다. '국보'·'보물'·'사적'은 각각 처음 실행될 때 한 번씩 계산돼 세 개의 캐시 항목으로 저장되고, 이후 같은 값의 재실행에서만 그 항목이 적중합니다. 그래서 부적절한 진술은 ④입니다.

①②③은 각각 결과 자체를 저장해 재집계를 생략한다는 점(①), 의존 객체 DML 커밋에 의한 캐시 무효화(②), 결과 캐시가 커서 공유와 별개 계층이라 캐시 적중에도 소프트파싱은 일어난다는 점(③)을 옳게 서술합니다.

선택지	판정	이유
①	○	결과 캐시는 집계 결과값을 Result Cache 영역에 저장해, 동일 질의의 다음 실행에서 블록 재읽기·재집계 없이 저장된 결과를 돌려줍니다
②	○	의존 객체(보수공사내역)에 DML이 커밋되면 관련 캐시 항목이 무효화되어, 다음 실행에서 재계산·재캐싱됩니다
③	○	커서 공유는 파싱된 커서(실행계획) 재사용, 결과 캐시는 결과값 재사용으로 계층이 다릅니다 — 캐시가 적중해도 커서를 찾는 소프트파싱은 수행됩니다
④	×	결과 캐시 키는 바인드 값을 포함하므로 :b1이 '국보'·'보물'로 다르면 캐시 항목이 걸립니다. 텍스트 동일만으로 한 번만 계산된다고 볼 수 없습니다

▪ 이 문제의 핵심

1. 결과 캐시의 키는 텍스트 + 바인드 값입니다. 바인드 값이 다르면 결과가 다르므로 별도 캐시 항목으로 저장됩니다 — 텍스트가 같다고 하나로 묶이지 않습니다.
2. 결과 캐시와 커서 공유는 다른 계층입니다. 커서 공유는 실행계획(라이브러리 캐시)을, 결과 캐시는 계산된 결과값을 재사용하며, 캐시 적중에도 소프트파싱은 일어납니다.
3. 무효화는 결과 캐시의 방어쇠입니다. 의존 객체에 DML이 커밋되면 관련 항목이 무효화되어 다음 실행에서 재계산됩니다.
4. 결과 캐시가 재집계를 생략합니다. 적중 시 블록을 다시 읽거나 SUM을 재수행하지 않고 저장된 값을 반환해, 반복 집계 부하를 줄입니다.

▪ 한 줄 정리 결과 캐시 항목은 SQL 텍스트에 더해 바인드 값까지 키로 삼아 값별로 걸리므로, "텍스트만 같으면 바인드가 달라도 한 번만 계산해 재사용한다"는 ④는 정규화 가정의 오류라 부적절합니다.

문제 14. 정답 ③

CBO의 추정치는 단일 테이블 선택도와 조인 카디널리티가 서로 다른 축이고, 조인 결과 건수는 조인 컬럼의 값 분포(특히 구별 값 수, NDV)에 크게 기댑니다.

단일 조건 선택도 : 한 테이블의 필터 결과 비율 (컬럼 히스토그램 · NDV가 좌우)

조인 카디널리티 : 대략 (card1 × card2) / (조인 컬럼 NDV 기반 조인 선택도)

⇒ 각 테이블 선택도가 정확해도 조인 컬럼 통계(NDV)가 어긋나면 조인 결과 건수 추정이 빗나감

③은 조인 카디널리티가 두 집합의 카디널리티 + 조인 컬럼의 값 분포로 추정되고, 조인 컬럼 NDV가 정확도를 좌우한다는 원리를 바르게 서술합니다. 단일 조건 선택도라는 축과 조인 카디널리티라는 축이 별개임을 정확히 구분했습니다.

나머지는 서로 독립인 두 축을 인과로 잘못 묶은 별개 축 혼동입니다.

- ①: 선택도가 조건식에서, 카디널리티가 테이블 통계에서 나온다는 산출 경로는 맞지만 그렇다고 두 값이 독립인 것은 아닙니다. 카디널리티 $\approx$ 선택도 $\times$ 전체 행 수라서 둘은 곧바로 연결되므로, 별개 축이 아닌 것을 별개로 본 혼동입니다.
- ②: 히스토그램(단일 컬럼 분포 축)이 단일 조건 선택도를 돕는 것은 맞으나, 조인 카디널리티 정확도와 조인 방식·조인 순서 결정(별개 축)까지 따라온다고 묶었습니다. 조인 결과 건수는 조인 컬럼 NDV의 축이고, 조인 방식·순서는 비용 비교의 축입니다.
- ④: 통계 최신성(추정 정확도 축)과 파싱 시간(파싱 비용 축)을 인과로 이었습니다. 통계가 정확해도 옵티마이저가 검토하는 후보 계획 수가 줄지는 않고, 최신 통계는 계획 품질에 관여할 뿐 파싱 시간을 줄이지 않아 두 축을 뒤섞은 혼동입니다.

선택지	판정	이유
①	×	산출 경로가 달라도 카디널리티 $\approx$ 선택도 $\times$ 전체 행 수라 둘은 직접 연결됩니다. '정해진 산출 관계가 없다'는 연결된 축을 독립으로 본 별개 축 혼동입니다
②	×	히스토그램은 단일 컬럼 선택도를 돕는 축이며, 조인 카디널리티 정확도는 조인 컬럼 NDV의 축, 조인 방식·조인 순서 결정은 비용 비교의 축입니다 — 하나가 나머지를 자동으로 끌어오지 않습니다
③	○	조인 카디널리티는 두 집합 카디널리티와 조인 컬럼 값 분포(NDV)로 추정되어, 단일 선택도가 정확해도 조인 컬럼 통계가 어긋나면 조인 결과 건수 추정이 빗나갑니다
④	×	통계가 정확해도 옵티마이저가 비교하는 후보 계획 수는 줄지 않으며, 최신 통계는 계획 품질(추정 정확도)에 관여할 뿐 하드파싱 시간을 줄이지 않습니다. 정확도 축과 파싱 비용 축을 인과로 묶은 혼동입니다

■ 이 문제의 핵심

1. 선택도와 카디널리티는 연결된 지표다. 카디널리티 $\approx$ 선택도 $\times$ 전체 행 수라 독립이 아닙니다.
2. 조인 카디널리티는 조인 컬럼 값 분포(NDV)가 좌우한다. 단일 조건 선택도가 정확해도 조인 컬럼 통계가 어긋나면 조인 결과 건수 추정이 빗나갑니다.
3. 히스토그램은 단일 컬럼 선택도의 축이다. 조인 카디널리티 정확도·조인 방식 최적화라는 다른 축을 자동으로 보장하지 않습니다.
4. 통계 최신성은 계획 품질의 축이다. 파싱 시간을 줄이는 것과는 별개 축입니다.

■ 한 줄 정리 조인 카디널리티는 두 집합 카디널리티와 조인 컬럼 값 분포(NDV)로 추정되어 단일 선택도가 정확해도 빗나갈 수 있다는 ③이 옳고, 선택도·카디널리티를 독립으로 보거나 히스토그램·통계 최신성을 다른 축의 결과까지 끌어오는 진술은 별개 축 혼동입니다.

문제 15. 정답 ④

Direct Path Insert의 인덱스 유지 방식은 일반 인서트와 다릅니다. 일반(conventional) 인서트는 행을 넣을 때마다 인덱스 리프를 그때그때 갱신하지만, direct path는 적재분에 대한 인덱스 엔트리를 임시 세그먼트에 따로 모아 두었다가 적재 종료 시점에 기존 인덱스와 한 번에 병합(merge) 합니다.

[인덱스 유지 방식 비교]

일반 인서트 : 행 1건 → 인덱스 3개 리프에 즉시 반영 (건별 $\times$ 인덱스 수)
 Direct Path : 적재분 인덱스 엔트리를 임시 세그먼트에 축적 → 종료 시 일괄 병합
 대량 + 다중 인덱스 : unusable 후 적재 → 인덱스 rebuild 가 병합 비용보다 유리한 경우가 많음

④는 두 군데를 뒤집었습니다. 첫째, direct path가 인덱스를 "행별로 즉시 반영"한다고 했으나 실제로는 일괄 병합 방식입니다. 둘째, 그래서 "unusable 후 재생성이 이득이 없다"고 했으나, 인덱스가 여럿이고 적재량이 클수록 인덱스를 미리 unusable로 두고 적제한 뒤 rebuild 하는 편이 병합 부하를 피해 더 빠른 경우가 많습니다. ④는 인덱스 유지 메커니즘과 그에 따른 튜닝 방향을 정반대로 서술했으므로 부적절합니다.

- ①은 direct path의 HWM 위 적재·버퍼 캐시 우회를, ②는 nologging이 direct path에서만 redo를 줄인다는 전제 조건을, ③은 병렬 DML의 PX별 적재와 테이블 배타 잠금을 옳게 서술합니다.

선택지	판정	이유
①	○	APPEND는 direct path로 HWM 위 새 블록에 기록하고 버퍼 캐시·freelist를 우회하므로 대량 적재가 빨라집니다
②	○	nologging + non-force-logging이라야 direct path의 데이터 redo가 최소화되며, 일반 인서트에는 nologging이 redo 절감 효과가 없습니다
③	○	병렬 DML을 켜면 PX 서버가 각자 익스텐트에 direct path로 적재하고, 대상 테이블은 배타 모드로 잠겨 커밋 전까지 다른 DML이 대기합니다
④	×	direct path는 인덱스 엔트리를 임시 세그먼트에 모아 종료 시 일괄 병합합니다. 다중 인덱스·대량 적재에선 unusable 후 rebuild가 유리한 경우가 많아 "이득 없음"은 반대입니다

■ 이 문제의 핵심

1. Direct Path Insert는 HWM 위 새 블록에 적재합니다. 버퍼 캐시와 freelist 탐색을 우회해 대량 적재를 가속합니다.
2. nologging은 direct path에서만 효과입니다. 세그먼트 NOLOGGING + DB non-force-logging 조건에서 데이터 redo가 최소화되며, 일반 인서트에는 무의미합니다.
3. 병렬 DML은 테이블을 배타 잠금합니다. PX 서버가 각자 익스텐트에 적재하고, 커밋 전까지 같은 테이블의 다른 DML은 대기합니다.
4. 인덱스는 일괄 병합 방식입니다. 행별 즉시 반영이 아니라 임시 세그먼트 축적 후 병합이라, 다중 인덱스·대량 적재에선 unusable 후 rebuild가 흔히 더 빠릅니다.

■ 한 줄 정리 Direct Path Insert의 인덱스는 임시 세그먼트에 모았다가 일괄 병합하며 대량·다중 인덱스에선 unusable 후 rebuild가 유리한데, ④는 "행별 즉시 반영이라 재생성이 이득 없다"고 정반대로 서술해 부적절합니다.

문제 16. 정답 ④

INTERVAL RANGE 파티셔닝에서 프루닝은 파티션 키(발행일자)에 가공이 없는 조건일 때 경계값과 대조해 걸리고, 인터벌 파티션은 미리 만들어 두는 게 아니라 그 구간의 첫 행이 insert될 때 자동 생성됩니다.

[조회·생성별 판정]

- ① 생성 시점에 12개월 사전 생성 → INTERVAL은 첫 insert 때 on-demand 생성 (사전 생성 아님)
 ② TO_CHAR(발행일자, 'YYYYMM')='202603' → 키를 형변환·가공 → 경계 대조 불가 → 전 파티션 스캔
 ③ 발행금액 >= 100000 → 발행금액은 파티션 키가 아님 → RANGE 프루닝 없음
 ④ 발행일자 >= 2026-03-01 AND < 2026-04-01 → 키에 가공 없는 범위 → 3월 파티션만 접근 / 첫 insert 시 자동 생성 ○

④는 파티션 키인 발행일자를 가공 없이 범위로 걸었습니다. 경계값(2026-03-01 ~ 2026-04-01)과 대조되어 3월 구간 파티션만 접근하고, 그 파티션이 아직 없더라도 해당 월 데이터가 처음 들어올 때 **NUMTOYMINTERVAL(1,'MONTH')** 규칙에 따라 자동 생성됩니다. 프루닝과 인터벌 자동 생성을 함께 정확히 서술했으므로 ④가 적절합니다.

나머지는 어긋납니다. ①은 **NUMTOYMINTERVAL(1,'MONTH')**을 사전 생성 규칙으로 읽었지만, DDL이 실제로 만드는 파티션은 **p_init** 하나뿐이고 이후 월 파티션은 첫 insert 때 필요분만 만들어집니다(그래서 DDL을 손댈 필요가 없다는 결론만 우연히 맞습니다). ②는 **TO_CHAR(발행일자, 'YYYYMM')='202603'**으로 키를 형변환·가공해, 3월을 겨냥한 듯 보여도 경계 대조가 불가능해 2억 6천만 건의 전 파티션을 스캔합니다 — "3월만 읽는다"는 부분진실입니다. ③은 **PARTITION BY RANGE (발행일자)**이므로 프루닝 기준은 발행일자뿐이고, 발행금액은 NUMBER 범위 조건이라 파티션 키가 아닌 일반 필터라 RANGE 프루닝이 걸리지 않습니다.

선택지	판정	이유
①	×	DDL이 실제로 만드는 파티션은 p_init 하나뿐이고, NUMTOYMINTERVAL(1,'MONTH') 은 사전 생성 규칙이 아니라 해당 구간의 첫 행이 insert될 때 on-demand로 파티션을 만들라는 규칙입니다 — "12개월치 사전 생성"이 틀렸습니다
②	×	발행일자가 파티션 키인 것은 맞지만 TO_CHAR(발행일자, 'YYYYMM') 은 그 키를 형변환·가공한 것이라 경계값과 대조할 수 없어 2억 6천만 건의 전 파티션을 스캔합니다 — "3월 파티션만 읽음"은 부분진실입니다
③	×	PARTITION BY RANGE (발행일자) 이므로 프루닝 기준은 발행일자뿐이고, 발행금액은 NUMBER 범위 조건이라 파티션 키가 아닌 일반 필터입니다. RANGE 파티셔닝이라는 사실만으로 금액 조건이 파티션을 줄이지는 못합니다
④	○	발행일자에 가공 없는 범위 조건이라 3월 구간 파티션만 접근하고, 없던 파티션은 첫 insert 시 인터벌 규칙으로 자동 생성됩니다

■ 이 문제의 핵심

1. RANGE 프루닝은 파티션 키에 가공이 없어야 경계값과 대조되어 걸립니다. 발행일자를 가공 없는 범위로 걸면 해당 구간 파티션만 읽습니다.

2. INTERVAL 파티션은 on-demand 생성입니다. 미리 만들어 두는 게 아니라 그 구간의 첫 행이 insert될 때 규칙에 따라 자동 생성됩니다.
3. 파티션 키가 아닌 컬럼(발행금액)은 프루닝 대상이 아닙니다. 범위 조건이라도 파티션을 줄이지 못하는 일반 필터입니다.
4. 키를 형변환·가공(TO_CHAR) 하면 경계 대조가 불가능해 전 파티션을 스캔합니다 — "월을 겨냥"이라는 걸모습이 부분진실 함정입니다.
- 한 줄 정리 발행일자를 가공 없는 범위로 건 ④만 3월 파티션 프루닝과 인터벌 자동 생성을 모두 충족하며, 사전 생성 오해(①)·키 형변환(②)·비키 필터(③)는 프루닝이 성립하지 않아 부적절합니다.

문제 17. 정답 ②

분배 방식은 어느 입력이 **PX SEND**를 거치고 그 SEND의 종류가 무엇인지로 읽습니다.

```
Id 6  PX SEND HASH :TQ10000 (Q1,00 → 해시 재분배)   급전지시를 발전기ID 해시로 뿌림
Id 8  TABLE ACCESS FULL 급전지시 (약 4.1억 건)       재분배 대상 - 대형
Id 10 PX SEND HASH :TQ10001 (Q1,01 → 해시 재분배)   발전기실적을 발전기ID 해시로 뿌림
Id 12 TABLE ACCESS FULL 발전기실적 (약 2.8억 건)     재분배 대상 - 대형
```

조인 입력 양쪽 모두 위에 **PX SEND HASH**가 있습니다. 급전지시는 Id 6, 발전기실적은 Id 10에서 각각 조인 키 발전기ID의 해시로 재분배됩니다.

두 입력이 같은 해시 함수로 뿌려지므로, 같은 발전기ID는 같은 서버로 모입니다. 각 서버는 자기 버킷 범위의 행만 받아 로컬로 조인(Id 4 **HASH JOIN BUFFERED**)합니다.

이 조합이 hash-hash 분배입니다. 두 테이블이 모두 대형(4.1억·2.8억)이라 한쪽을 복제(broadcast)하면 그 큰 집합이 서버 수만큼 불어나므로, 양쪽을 해시로 갈라 나눠 갖는 편이 유리합니다.

- ① broadcast : 한쪽에 PX SEND BROADCAST, 다른 쪽 SEND 없음 → 양쪽 다 PX SEND HASH
- ② hash-hash : 양쪽 위에 각각 PX SEND HASH → Id 6·Id 10과 일치
- ③ full PWJ : 두 입력이 같은 PX PARTITION 아래, SEND 없음 → 양쪽에 SEND HASH 있음
- ④ partial PWJ : 한쪽에 PX SEND PARTITION (KEY) → SEND 종류가 HASH

양쪽 입력에 **PX SEND HASH**가 하나씩(Id 6·Id 10) 있다는 사실이 broadcast·full PWJ·partial PWJ를 한꺼번에 배제합니다. 따라서 ②가 옳습니다.

선택지	판정	이유
①	×	broadcast면 한쪽에 PX SEND BROADCAST 가 있고 다른 쪽은 SEND가 없어야 하는데, 이 플랜은 Id 6·Id 10 양쪽 모두 PX SEND HASH 입니다. 분배 방식을 hash-hash에서 broadcast로 뒤바꾼 진술입니다
②	○	조인 입력 양쪽(Id 8·Id 12) 위에 각각 PX SEND HASH (Id 6·Id 10)가 있어 발전기ID 해시로 양쪽을 재분배하고, 각 서버가 자기 버킷 범위만 로컬 조인하는 hash-hash 분배와 일치합니다
③	×	full 파티션 와이즈면 두 입력이 같은 PX PARTITION 아래에서 SEND 없이 읽혀야 하는데, 양쪽이 PX SEND HASH (Id 6·Id 10)로 재분배되고 있고 두 테이블 모두 발전기ID 기준 파티션도 아닙니다 — hash-hash를 PWJ로 뒤바꾼 값스왑입니다
④	×	partial 파티션 와이즈면 재분배되는 쪽에 PX SEND PARTITION (KEY) 가 있어야 하는데, Id 6·Id 10의 SEND는 둘 다 HASH 입니다. SEND 종류를 HASH에서 PARTITION (KEY)로 뒤바꾼 진술입니다

▪ 이 문제의 핵심

1. 분배 방식은 조인 입력의 **PX SEND** 종류로 판별합니다. 여기서는 양쪽 입력에 **PX SEND HASH**(Id 6·Id 10)가 하나씩 있습니다.
2. hash-hash는 대형×대형에서 양쪽을 조인 키 해시로 나눠 갖는 방식입니다. 같은 발전기ID가 같은 서버로 모여 각 서버가 자기 버킷만 조인합니다.
3. 양쪽 모두 SEND HASH라는 점이 핵심 단서입니다. 한쪽만 SEND가 있으면 broadcast, SEND가 아예 없으면 full PWJ입니다.
4. broadcast·full PWJ·partial PWJ는 각각 **PX SEND BROADCAST**·같은 PX PARTITION 아래·**PX SEND PARTITION (KEY)**가 있어야 성립합니다. 이 플랜의 SEND는 양쪽 HASH뿐이라 셋 다 배제됩니다 — 분배 방식을 뒤바꾼 값스왑 오답입니다.

- 한 줄 정리 조인 입력 양쪽(Id 8·Id 12) 위에 각각 **PX SEND HASH**(Id 6·Id 10)가 있어 발전기ID 해시로 두 대형 테이블을 모두 재분배하고 각 서버가 자기 버킷만 로컬 조인하므로, hash-hash 분배다.

문제 18. 정답 ②

윈도우 프레임의 **ROWS**와 **RANGE**는 범위를 세는 기준이 서로 다른 하위 방식입니다.

ROWS : 물리적 '행 개수'로 프레임 경계를 셈 (예: 앞 2행 ~ 현재 행 = 정확히 3행)

RANGE : 정렬 값(ORDER BY 값)을 기준으로 경계를 셈 (같은 값의 동순위 행은 한 묶음으로 함께 포함)

⇒ ORDER BY 컬럼에 동일 값(tie)이 있으면 두 방식의 프레임 구성 행이 달라져 결과가 같릴 수 있음

②는 **ROWS**의 성질(물리적 행 개수로 센다)을 **ROWS**와 **RANGE** 둘 전체의 성질인 양 끌어 올려, 두 절을 바꿔 써도 결과가 같다고 했습니다. 실제로 **RANGE**는 물리적 행 수가 아니라 정렬 값으로 경계를 잡아, 정렬 컬럼에 같은 값이 여럿이면 동순위 행을 한꺼번에 프레임에 넣습니다. 그래서 동순위가 있을 때 **ROWS**와 **RANGE**는 프레임 구성이 달라져 결과가 같릴 수 있습니다. 한 하위 방식의 성질을 상위 범주 전체로 넓힌 하위항목 전체화라 ②가 부적절합니다.

①③④는 모두 옳습니다.

①: **UNION**은 중복 제거를 위해 SORT UNIQUE/HASH UNIQUE가 따르고, **UNION ALL**은 중복 제거가 없어 그 정렬·해시 없이 이어 붙입니다.

③: **RANK**는 동순위 뒤 순위를 건너뛰고, **DENSE_RANK**는 건너뛰지 않으며, **ROW_NUMBER**는 동순위여도 유일 번호를 매깁니다.

④: **INTERSECT·MINUS**도 중복 제거를 수반해 정렬(또는 해시)이 따르고, 유일 집합을 반환합니다.

선택지	판정	이유
①	○	UNION 은 중복 제거로 SORT UNIQUE/HASH UNIQUE가 따르고, UNION ALL 은 중복 제거가 없어 정렬·해시 없이 두 집합을 이어 붙입니다
②	×	RANGE 는 물리적 행 수가 아니라 정렬 값으로 경계를 잡아 동순위 행을 함께 넣으므로, 동순위가 있으면 ROWS 와 결과가 같립니다. ROWS 의 성질을 둘 전체로 넓힌 하위항목 전체화입니다
③	○	RANK 는 동순위 뒤를 건너뛰고(1,1,3) DENSE_RANK 는 건너뛰지 않으며(1,1,2), ROW_NUMBER 는 동순위여도 유일 번호를 줍니다
④	○	INTERSECT·MINUS 도 중복 제거를 수반해 정렬(또는 해시)이 따르고, 중복이 제거된 유일 집합을 반환합니다

▪ 이 문제의 핵심

1. **ROWS**와 **RANGE**는 경계 기준이 다르다. **ROWS**는 물리적 행 개수, **RANGE**는 정렬 값 기준이라 동순위가 있으면 프레임 구성이 같립니다 — 바꿔 써도 같다고 볼 수 없습니다.

2. **UNION**은 중복 제거로 정렬을 부른다. SORT UNIQUE/HASH UNIQUE가 따르고, **UNION ALL**은 그 작업 없이 이어 붙입니다.

3. 순위 함수는 동순위 처리가 같린다. **RANK**는 건너뛰고, **DENSE_RANK**는 건너뛰지 않으며, **ROW_NUMBER**는 유일 번호를 줍니다.

4. **INTERSECT·MINUS**도 중복을 제거한다. **UNION**과 마찬가지로 정렬(또는 해시)을 수반해 유일 집합을 반환합니다.

▪ 한 줄 정리 **RANGE**는 정렬 값으로 프레임 경계를 잡아 동순위 행을 함께 넣으므로 **ROWS**와 결과가 같릴 수 있는데, "둘 다 물리적 행 개수로 세니 바꿔 써도 같다"는 ②는 **ROWS**의 성질을 상위 범주 전체로 넓힌 하위항목 전체화라 부적절합니다.

문제 19. 정답 ①

오라클의 기본 격리수준 READ COMMITTED에서 읽기 일관성은 문장 수준(statement-level)입니다. 즉 각 SELECT는 자기 문장이 시작되는 시점의 SCN 스냅샷을 기준으로 읽으며, 같은 트랜잭션 안이라도 문장이 다르면 스냅샷이 새로 잡힙니다. 트랜잭션 수준 일관성(같은 트랜잭션 내내 한 스냅샷 고정)은 SERIALIZABLE에서만 성립합니다.

[TX2 두 SELECT 값 계산]

t1 첫 SELECT : 문장 시작 스냅샷 = 초기값

가=120000, 나=240000, 다=360000 → SUM = 720000

t2 TX1 COMMIT : 나=300000, 다=400000 확정

t3 둘째 SELECT : READ COMMITTED → 문장 시작 시점 새 스냅샷 (t3 > t2 커밋)

가=120000, 나=300000, 다=400000

SUM = 120000 + 300000 + 400000 = 820000

두 번째 SELECT는 t3에 새로 시작하며, 그 시점엔 TX1이 t2에 이미 커밋해 두었으므로 갱신된 최신 커밋값(나 300000·다 400000)을 봅니다. 따라서 SUM = 820,000, 정답은 ①입니다. 만약 TX2가 SERIALIZABLE이었다면 두 SELECT 모두 트랜잭션 시작 스냅샷을 써 720,000으로 같았겠지만, 기본 READ COMMITTED라 문장마다 다시 읽습니다.

②③④는 스냅샷값과 부분 갱신값을 섞어 만든 값스왑 오답입니다. ②는 나만 갱신값(300000)으로 섞은 780,000, ③은 다만 갱신값(400000)으로 섞은 760,000, ④는 문장 재스냅샷을 놓쳐 첫 SELECT와 같다고 본 720,000입니다.

선택지	판정	이유
①	○	READ COMMITTED는 문장 수준 일관성이라 t3 둘째 SELECT가 새 스냅샷으로 최신 커밋값을 읽어 120000+300000+400000=820000입니다
②	×	120000+300000+360000. 나만 갱신값, 다는 원래값으로 섞은 값스왑입니다
③	×	120000+240000+400000. 다만 갱신값, 나는 원래값으로 섞은 값스왑입니다
④	×	720000은 첫 SELECT의 스냅샷 값입니다. 문장 수준 일관성상 둘째 SELECT는 재스냅샷하므로 이 값이 아닙니다(트랜잭션 수준은 SERIALIZABLE에서만)

■ 이 문제의 핵심

1. READ COMMITTED는 문장 수준 읽기 일관성입니다. 각 SELECT가 자기 문장 시작 SCN으로 스냅샷을 새로 잡아, 같은 트랜잭션이라도 문장마다 최신 커밋값을 다시 봅니다.
2. 트랜잭션 수준 고정은 SERIALIZABLE입니다. 같은 스냅샷을 트랜잭션 내내 유지하려면 격리수준을 올려야 하며, 그때만 두 SELECT가 720,000으로 같습니다.
3. MVCC는 커밋 순서를 반영합니다. t2 커밋이 t3 문장 시작보다 앞서므로, 둘째 SELECT는 300,000·400,000을 봅니다.
4. Undo는 CR 재구성용 자료입니다. 문장 시작 SCN보다 최신인 변경 블록은 Undo로 되감아 읽지만, 여기서 t3 스냅샷이 t2 커밋 이후라 되감을 것이 없어 갱신값 그대로 읽습니다. (긴 단일 문장이 스냅샷보다 오래돼 Undo가 재활용되면 ORA-01555가 나는 것과 대비됩니다.)

■ 한 줄 정리 READ COMMITTED의 문장 수준 일관성상 TX2의 둘째 SELECT는 t3에 재스냅샷해 t2에 커밋된 최신값을 읽으므로 SUM=820,000(①)이며, 부분 조합(780,000·760,000)이나 첫 스냅샷(720,000)은 값스왑 오답입니다.

문제 20. 정답 ③

RCSI(낙관적 동시성 제어)는 읽기에만 행 버전을 적용합니다. READ COMMITTED에서 SELECT는 공유 잠금을 걸지 않고 버전 저장소(tempdb)의 마지막 커밋된 행 버전을 읽으므로, 미커밋 X 잠금을 쥔 갱신 세션을 기다리지 않습니다. 반면 쓰기끼리는 여전히 잠금으로 직렬화됩니다 — 같은 행을 갱신하려는 두 세션은 X 잠금으로 경합합니다.

[표 판독]

52 : X GRANT (미커밋) → 갱신 중, X 보유
 67 : 잠금 요청 없음 → 버전 읽기 → 52를 기다리지 않음 (블로킹 없음)
 58 : X WAIT (blocked by 52) → 같은 행 UPDATE → X 경합 → 대기

읽기(67) : 버전으로 우회 → 대기 없음

쓰기(58) : 버전 대상 아님 → X 잠금 경합 → 52 커밋까지 대기

③은 표를 정확히 읽었습니다. 67은 **request_status**에 잠금 자체가 없어(버전 읽기) 52를 기다리지 않고, 58은 **X WAIT**로 52에 막혀 있습니다. RCSI가 읽기 블로킹은 없애되 쓰기 경합은 그대로 둔다는 원리와 일치하므로, 적절한 해석은 ③입니다. 나머지는 어긋납니다. ①은 RCSI가 SELECT에 X 잠금을 걸어 UPDATE를 차단한다고 했으나, 표의 67은 잠금 요청이 없고 낙관적 읽기는 잠금을 걸지 않습니다 — 방향이 반대입니다. ②는 UPDATE끼리도 버전으로 동시 갱신된다고 했지만 행 버전은 읽기에만 적용되며, 표의 58은 실제로 **X WAIT**로 52에 막혀 대기 중입니다. ④는 "둘 다 tempdb를 쓴다"는 사실을 낙관적 제어의 증거로 삼았는데, 이는 RCSI 세션에 공통으로 나타나는 단서를 판별 근거로 착각한 것이고, 실제로 58은 잠금 대기 중이라 "잠금 대기가 없어야 정상"이라는 결론도 표와 어긋납니다.

선택지	판정	이유
①	×	낙관적 읽기는 잠금을 걸지 않습니다. 표의 67은 잠금 요청이 없으며, SELECT가 X를 걸어 52를 차단한다는 것은 방향이 반대입니다
②	×	행 버전은 읽기에만 적용됩니다. 쓰기끼리는 X로 직렬화되며, 표의 58은 실제로 52에 막혀 X WAIT 상태입니다
③	○	67은 잠금 없이 버전을 읽어 블로킹이 없고, 58은 X 잠금으로 52와 경합해 대기합니다 — 표의 세 행 그대로입니다
④	×	tempdb 공동 사용은 RCSI 세션에 공통인 단서라 판별 근거가 못 됩니다. 표의 58은 X WAIT 라 "잠금 대기 없음"도 성립하지 않습니다

■ 이 문제의 핵심

1. RCSI는 읽기에만 행 버전을 적용합니다. READ COMMITTED의 SELECT가 공유 잠금 대신 버전 저장소의 마지막 커밋 버전을 읽어, 갱신 세션을 기다리지 않습니다.
2. 쓰기끼리는 여전히 잠금 경합합니다. 같은 행을 갱신하려는 두 세션은 X 잠금으로 직렬화되어 한쪽이 대기합니다(58↔52).

3. 읽기 블록킹은 사라지되 쓰기 블록킹은 남습니다. RCSI의 효과는 "읽기가 쓰기를, 쓰기가 읽기를 막지 않는" 데 있지 쓰기 경합 제거가 아닙니다.
4. "tempdb를 함께 쓴다"는 공통 단서입니다. RCSI 세션 모두에 해당하므로 그 자체로 특정 세션의 상태를 판별하지 못합니다 — 판별은 **request_status**(잠금/버전)로 해야 합니다.
- 한 줄 정리 RCSI는 읽기(67)를 버전으로 우회시켜 블록킹을 없애되 쓰기끼리(58↔52)의 X 경합은 그대로 두므로 ③이 적절하며, "tempdb 공용 = 낙관적 제어 증거"(④)는 공통 단서를 판별 근거로 착각한 오독입니다.